Guía Completa Definitiva v5.0 — Todo Corregido

Guía ÚNICA y COMPLETA para construir la app de llamadas VoIP y mensajes desde cero. Incluye el google-services.json real, todos los archivos Kotlin y XML, y todas las correcciones de las 3 auditorías anteriores (21 bugs corregidos).

21 bugs corregidos	**Google JSON incluido**	**100% auto-contenida**
Auth verificación email	**WebRTC sin crashes**	**Teclado siempre visible**
Contactos + permisos mic	**ICE candidates sin dup.**	**Historial se actualiza**
Ticks de lectura	**Mensajes programados**	**Config notificaciones**

Datos del Proyecto Firebase

Project ID	`app-de-llamada`
Project Number	`717516447001`
Package Name	`com.simplecallapp`
App ID	`1:717516447001:android:f62e5964b9c7035a0db988`
Web Client ID	`717516447001-koosm55a97p2h1hu7h87oe3id2rvhf1r.apps.googleusercontent.com`
SHA-1	`6b927e6d9332018d615e70927203623734649fa4`
API Key	`AIzaSyAb3dF0OBwknGFxEezknHbMAJZ6WbG5njo`
Storage Bucket	`app-de-llamada.firebasestorage.app`

Versión 5.0 Final — Mayo 2025

Índice Completo

1. Stack tecnológico y por qué
2. google-services.json — tu archivo real listo para usar
3. Crear proyecto en Android Studio
4. Configuración Firebase Console (paso a paso con tus datos)
    - Authentication
    - Firestore + índices obligatorios
    - FCM
5. Gradle — build.gradle proyecto y app
6. Logo vectorial — ic_launcher
7. Tema, colores y dimensiones Material 3
8. AndroidManifest.xml completo
9. Estructura de carpetas
10. Modelos de datos
11. Autenticación CORREGIDA
    - AuthRepository
    - AuthViewModel
    - LoginActivity
    - ChooseNumber
12. PermissionActivity — permisos al inicio
13. Repositorios
    - UserRepository
    - ChatRepository
    - ContactRepository
    - CallHistoryRepository
14. WebRTC CORREGIDO — sin crash de doble inicialización
15. WebRTCSignaling CORREGIDO — solo ICE ADDED
16. CallForegroundService — ringtone configurable
17. CallActivity CORREGIDO — cleanupScope
18. MyFirebaseMessagingService
19. Adapters CORREGIDOS
    - MessageAdapter con tvTick
    - ContactsAdapter
    - CallHistoryAdapter
20. Fragments — Llamadas CORREGIDOS
    - CallsFragment
    - DialFragment CORREGIDO
    - CallHistoryFragment
21. Fragments — Mensajes CORREGIDOS

- ConversationsFragment
- ChatFragment CORREGIDO

22. ContactsFragment CORREGIDO — permiso RECORD_AUDIO
23. ProfileFragment + Logout + Cambiar contraseña
24. NotifSettingsFragment
25. MessageSettingsFragment CORREGIDO
26. CallSettingsFragment CORREGIDO
27. SendMessageWorker CORREGIDO
28. MainActivity + nav_graph + menus completos
29. SimpleCallApplication + strings + colors + themes + dimens
30. TODOS los layouts XML
    - activity_login
    - activity_choose_number
    - activity_permission
    - activity_main
    - activity_call
    - fragment_calls
    - fragment_dial CORREGIDO
    - fragment_call_history
    - fragment_conversations
    - fragment_chat
    - fragment_contacts
    - fragment_profile
    - fragment_notif_settings
    - fragment_message_settings
    - fragment_call_settings
    - items (sent/received/contact/call_history/conversation)
    - circle_btn_bg
31. Reglas Firestore — con permiso de update para campo read
32. Generar APK paso a paso
33. Checklist final de pruebas

1. Stack Tecnológico

Backend — Firebase Plan Spark (gratis)

- Firebase Authentication — Google Sign-In + Email/Password con verificación.
- Cloud Firestore — mensajes en tiempo real, señalización WebRTC, contactos.
- Firebase Cloud Messaging (FCM) — notificaciones push. Gratis ilimitado.

Llamadas — WebRTC (io.github.webrtc-sdk:android:114.5.0)

- Maven Central. Codec Opus: ~30-40 kbps. Misma calidad que Discord/WhatsApp.
- UNIFIED_PLAN: audio remoto capturado correctamente en onTrack().
- Señalización via Firestore — sin servidor extra.

TURN Server — Open Relay (Metered.ca) — 20 GB/mes gratis

- Registro en https://app.metered.ca — sin tarjeta de crédito.
- Puertos 80/443 — pasa cualquier firewall. 99.999% uptime.

2. google-services.json — Tu Archivo Real

✓ Este es TU archivo google-services.json real. Copiarlo exactamente a la carpeta app/ del proyecto.

app/google-services.json — COPIAR EN app/ (NO en la raíz)


```
{
"project_info": {
"project_number": "717516447001",
"project_id": "app-de-llamada",
"storage_bucket": "app-de-llamada.firebasestorage.app"
},
"client": [
{
"client_info": {
"mobilesdk_app_id": "1:717516447001:android:f62e5964b9c7035a0db988",
"android_client_info": {
"package_name": "com.simplecallapp"
}
},
"oauth_client": [
{
"client_id": "717516447001-9djcjlb3mieed67gcomc21idglh3tqiv.apps.googleusercontent.com",
"client_type": 1,
"android_info": {
"package_name": "com.simplecallapp",
"certificate_hash": "6b927e6d9332018d615e709272036237346 49fa4"
}
},
{
"client_id": "717516447001-koosm55a97p2h1hu7h87oe3id2rvhf1r.apps.googleusercontent.com",
"client_type": 3
}
],
"api_key": [
{ "current_key": "AIzaSyAb3dF0OBwknGFxEezknHbMAJZ6WbG5njo" }
],
"services": {
"appinvite_service": {
"other_platform_oauth_client": [
{
"client_id": "717516447001-koosm55a97p2h1hu7h87oe3id2rvhf1r.apps.googleusercontent.com",
"client_type": 3
}
]
}
}
}
],
"configuration_version": "1"
}
```

3. Crear el Proyecto en Android Studio

1. File → New → New Project → Empty Views Activity
2. Name: SimpleCallApp | Package name: com.simplecallapp | Language: Kotlin
3. Minimum SDK: API 23 (Android 6.0) | Build config: Groovy DSL
4. Finish → esperar Gradle sync inicial
5. Cambiar vista a 'Project' (dropdown arriba izquierda de Android Studio)
6. Copiar google-services.json (sección 2) a la carpeta app/
7. File → Sync Project with Gradle Files

4. Configuración Firebase Console

ℹ Tu proyecto Firebase ya existe: app-de-llamada. Solo verificar que los métodos de autenticación estén habilitados.

4.1 Authentication — Verificar que estén habilitados

1. console.firebase.google.com → proyecto app-de-llamada → Authentication
2. Pestaña Sign-in method → verificar que estén habilitados:
3. → Correo electrónico/contraseña: HABILITADO
4. → Google: HABILITADO

4.2 Firestore Database

1. Firestore Database → Crear base de datos (si no existe) → Modo producción
2. Ubicación: nam5 (us-central)

Índices compuestos OBLIGATORIOS — Firestore → Índices → Agregar índice compuesto:

Colección	Campo 1	Campo 2	Campo 3
messages	from (ASC)	to (ASC)	timestamp (ASC)
calls	receiverUid (ASC)	status (ASC)	—
contacts	ownerUid (ASC)	name (ASC)	—
call_history	callerUid (ASC)	timestamp (DESC)	—
call_history	receiverUid (ASC)	timestamp (DESC)	—

4.3 Cloud Messaging (FCM)

Firebase Console → Cloud Messaging → Comenzar. Sin configuración adicional.

5. Gradle

build.gradle (raíz del proyecto — NO en /app/)

```
buildscript {
repositories { google(); mavenCentral() }
dependencies {
classpath 'com.android.tools.build:gradle:8.2.2'
classpath 'org.jetbrains.kotlin:kotlin-gradle-plugin:1.9.22'
classpath 'com.google.gms:google-services:4.4.1'
}
}
allprojects {
repositories { google(); mavenCentral()
maven { url 'https://jitpack.io' } }
}
```

app/build.gradle

```
plugins {
id 'com.android.application'
id 'org.jetbrains.kotlin.android'
id 'com.google.gms.google-services'
}
android {
namespace 'com.simplecallapp'
compileSdk 34
defaultConfig {
applicationId 'com.simplecallapp'
minSdk 23; targetSdk 34
versionCode 1; versionName '1.0'
}
buildFeatures { viewBinding true }
compileOptions {
sourceCompatibility JavaVersion.VERSION_17
targetCompatibility JavaVersion.VERSION_17
}
kotlinOptions { jvmTarget = '17' }
packaging {
resources {
excludes += '/META-INF/{AL2.0,LGPL2.1}'
excludes += '/META-INF/INDEX.LIST'
excludes += '/META-INF/DEPENDENCIES'
}
}
}
dependencies {
implementation platform('com.google.firebase:firebase-bom:33.1.0')
implementation 'com.google.firebase:firebase-auth-ktx'
implementation 'com.google.firebase:firebase-firestore-ktx'
implementation 'com.google.firebase:firebase-messaging-ktx'
implementation 'com.google.android.gms:play-services-auth:21.2.0'
// WebRTC — Maven Central (no JCenter)
implementation 'io.github.webrtc-sdk:android:114.5.0'
implementation 'androidx.recyclerview:recyclerview:1.3.2'
```

```
implementation 'androidx.viewpager2:viewpager2:1.1.0'
implementation 'com.google.android.material:material:1.12.0'
implementation 'androidx.navigation:navigation-fragment-ktx:2.7.7'
implementation 'androidx.navigation:navigation-ui-ktx:2.7.7'
implementation 'androidx.activity:activity-ktx:1.9.0'
implementation 'androidx.fragment:fragment-ktx:1.7.1'
implementation 'androidx.lifecycle:lifecycle-viewmodel-ktx:2.8.2'
implementation 'androidx.lifecycle:lifecycle-livedata-ktx:2.8.2'
implementation 'org.jetbrains.kotlinx:kotlinx-coroutines-android:1.8.1'
implementation 'org.jetbrains.kotlinx:kotlinx-coroutines-play-services:1.8.1'
implementation 'androidx.work:work-runtime-ktx:2.9.0'
}
```

6. Logo Vectorial

res/drawable/ic_launcher_foreground.xml

```
<?xml version="1.0" encoding="utf-8"?>
<vector xmlns:android="http://schemas.android.com/apk/res/android"
android:width="108dp" android:height="108dp"
android:viewportWidth="108" android:viewportHeight="108">
<path android:fillColor="#FFFFFF"
android:pathData="M68,42 C64,38 58,36 54,40 L50,44 C48,46 44,46 41,43
L33,35 C30,32 30,28 32,26 L36,22 C40,18 38,12 34,8 L30,4 C26,0 20,2 18,6
C12,20 16,38 30,52 L42,64 C52,74 64,78 74,76 C80,74 86,68 88,62
C90,56 86,50 80,48 Z"
android:scaleX="0.72" android:scaleY="0.72"
android:translateX="20" android:translateY="18"/>
<path android:strokeColor="#FFFFFF" android:strokeWidth="4"
android:strokeLineCap="round" android:fillColor="@android:color/transparent"
android:pathData="M72,40 Q80,46 80,54" android:strokeAlpha="0.9"/>
<path android:strokeColor="#FFFFFF" android:strokeWidth="4"
android:strokeLineCap="round" android:fillColor="@android:color/transparent"
android:pathData="M78,34 Q92,44 92,58" android:strokeAlpha="0.65"/>
<path android:strokeColor="#FFFFFF" android:strokeWidth="4"
android:strokeLineCap="round" android:fillColor="@android:color/transparent"
android:pathData="M84,28 Q104,42 104,62" android:strokeAlpha="0.35"/>
</vector>
```

res/drawable/ic_launcher_background.xml

```
<?xml version="1.0" encoding="utf-8"?>
<shape xmlns:android="http://schemas.android.com/apk/res/android"
android:shape="rectangle">
<gradient android:startColor="#6D28D9" android:endColor="#4C1D95"
android:angle="135" android:type="linear"/>
</shape>
```

res/mipmap-anydpi-v26/ic_launcher.xml (y ic_launcher_round.xml — igual)

```
<?xml version="1.0" encoding="utf-8"?>
<adaptive-icon xmlns:android="http://schemas.android.com/apk/res/android">
<background android:drawable="@drawable/ic_launcher_background"/>
<foreground android:drawable="@drawable/ic_launcher_foreground"/>
</adaptive-icon>
```

7. Tema, Colores y Dimensiones

res/values/colors.xml

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
<color name="purple_700">#4C1D95</color>
<color name="purple_500">#6D28D9</color>
<color name="purple_300">#A78BFA</color>
<color name="purple_100">#EDE9FE</color>
<color name="green_500">#10B981</color>
<color name="red_500">#EF4444</color>
<color name="surface">#FFFFFF</color>
<color name="background">#F8F7FF</color>
<color name="on_surface">#1E1B4B</color>
<color name="text_secondary">#64748B</color>
<color name="call_bg">#0F0A1E</color>
<color name="tick_delivered">#9E9E9E</color>
<color name="tick_read">#2196F3</color>
</resources>
```

res/values/themes.xml

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
<style name="Theme.SimpleCallApp"
parent="Theme.Material3.DayNight.NoActionBar">
<item name="colorPrimary">@color/purple_500</item>
<item name="colorPrimaryVariant">@color/purple_700</item>
<item name="colorOnPrimary">@android:color/white</item>
<item name="colorSecondary">@color/green_500</item>
<item name="android:statusBarColor">@color/purple_700</item>
<item name="android:windowLightStatusBar">false</item>
</style>
<style name="Theme.SimpleCallApp.Call"
parent="Theme.Material3.Dark.NoActionBar">
<item name="colorPrimary">@color/purple_300</item>
<item name="android:statusBarColor">@color/call_bg</item>
</style>
</resources>
```

res/values/dimens.xml

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
<dimen name="card_radius">16dp</dimen>
<dimen name="card_elevation">4dp</dimen>
<dimen name="spacing_sm">8dp</dimen>
<dimen name="spacing_md">16dp</dimen>
<dimen name="spacing_lg">24dp</dimen>
</resources>
```

8. AndroidManifest.xml

app/src/main/AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
<uses-permission android:name="android.permission.INTERNET"/>
<uses-permission android:name="android.permission.RECORD_AUDIO"/>
<uses-permission android:name="android.permission.MODIFY_AUDIO_SETTINGS"/>
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE"/>
<uses-permission android:name="android.permission.WAKE_LOCK"/>
<uses-permission android:name="android.permission.VIBRATE"/>
<uses-permission android:name="android.permission.FOREGROUND_SERVICE"/>
<uses-permission android:name="android.permission.FOREGROUND_SERVICE_PHONE_CALL"/>
<uses-permission android:name="android.permission.POST_NOTIFICATIONS"/>
<application
android:name=".SimpleCallApplication"
android:allowBackup="true"
android:icon="@mipmap/ic_launcher"
android:label="@string/app_name"
android:roundIcon="@mipmap/ic_launcher_round"
android:supportsRtl="true"
android:theme="@style/Theme.SimpleCallApp">
<activity android:name=".ui.auth.LoginActivity" android:exported="true">
<intent-filter>
<action android:name="android.intent.action.MAIN"/>
<category android:name="android.intent.category.LAUNCHER"/>
</intent-filter>
</activity>
<activity android:name=".ui.auth.PermissionActivity" android:exported="false"/>
<activity android:name=".ui.auth.ChooseNumberActivity" android:exported="false"/>
<activity android:name=".ui.main.MainActivity" android:exported="false"/>
<activity android:name=".ui.call.CallActivity"
android:exported="false"
android:theme="@style/Theme.SimpleCallApp.Call"
android:showOnLockScreen="true"
android:turnScreenOn="true"/>
<service android:name=".service.MyFirebaseMessagingService"
android:exported="false">
<intent-filter>
<action android:name="com.google.firebase.MESSAGING_EVENT"/>
</intent-filter>
</service>
<service android:name=".service.CallForegroundService"
android:exported="false"
android:foregroundServiceType="phoneCall"/>
</application>
</manifest>
```

9. Estructura de Carpetas

java/com/simplecallapp/

```
■■■ SimpleCallApplication.kt
■■■ data/
■ ■■■ model/ User.kt Message.kt Contact.kt CallHistory.kt
■ ■■■ repository/
■ ■■■ AuthRepository.kt UserRepository.kt
■ ■■■ ChatRepository.kt ContactRepository.kt
■ ■■■ CallHistoryRepository.kt
■■■ ui/
■ ■■■ auth/ LoginActivity.kt PermissionActivity.kt
■ ■ ChooseNumberActivity.kt AuthViewModel.kt
■ ■■■ main/ MainActivity.kt
■ ■■■ calls/ CallsFragment.kt DialFragment.kt
■ ■ CallHistoryFragment.kt CallHistoryAdapter.kt
■ ■■■ chat/ ConversationsFragment.kt ChatFragment.kt MessageAdapter.kt
■ ■■■ contacts/ ContactsFragment.kt ContactsAdapter.kt
■ ■■■ call/ CallActivity.kt
■ ■■■ profile/ ProfileFragment.kt
■ ■■■ settings/ NotifSettingsFragment.kt MessageSettingsFragment.kt
■ CallSettingsFragment.kt
■■■ webrtc/ WebRTCClient.kt WebRTCSignaling.kt CallState.kt
■■■ service/ MyFirebaseMessagingService.kt CallForegroundService.kt
■■■ workers/ SendMessageWorker.kt

res/layout/ (17 archivos XML — todos en sección 30)
res/drawable/ ic_launcher_foreground.xml ic_launcher_background.xml circle_btn_bg.xml
res/mipmap-anydpi-v26/ ic_launcher.xml ic_launcher_round.xml
res/navigation/ nav_graph.xml
res/menu/ bottom_nav_menu.xml menu_chat.xml
res/values/ strings.xml colors.xml themes.xml dimens.xml
```

10. Modelos de Datos

data/model/User.kt

```
package com.simplecallapp.data.model
data class User(
val uid: String = "",
val email: String = "",
val displayName: String = "",
val phoneNumber: String = "",
val fcmToken: String = ""
)
```

data/model/Message.kt

```
package com.simplecallapp.data.model
data class Message(
val id: String = "",
val from: String = "",
val fromNumber: String = "",
val to: String = "",
val toNumber: String = "",
val text: String = "",
val timestamp: Long = 0L,
val scheduledAt: Long = 0L,
val delivered: Boolean = false,
val read: Boolean = false
)
```

data/model/Contact.kt

```
package com.simplecallapp.data.model
data class Contact(
val id: String = "",
val ownerUid: String = "",
val contactUid: String = "",
val name: String = "",
val number: String = ""
)
```

data/model/CallHistory.kt

```
package com.simplecallapp.data.model
enum class CallType { OUTGOING, INCOMING, MISSED }
data class CallHistory(
val id: String = "",
val callerUid: String = "",
val receiverUid: String = "",
val callerNumber: String = "",
val receiverNumber: String = "",
val type: String = CallType.OUTGOING.name,
val durationSeconds: Int = 0,
val timestamp: Long = 0L
)
```

11. Autenticación — Completamente Corregida

✓ Versión final que incorpora todas las correcciones de auth_fix.py: reloadUser() antes de isEmailVerified, estados separados, errores en español.

data/repository/AuthRepository.kt

```
package com.simplecallapp.data.repository
import com.google.firebase.auth.FirebaseAuth
import com.google.firebase.auth.FirebaseUser
import com.google.firebase.auth.GoogleAuthProvider
import kotlinx.coroutines.tasks.await
class AuthRepository {
private val auth = FirebaseAuth.getInstance()
fun getCurrentUser(): FirebaseUser? = auth.currentUser
fun isLoggedIn(): Boolean = auth.currentUser != null
suspend fun registerWithEmail(email: String, pass: String): Result<FirebaseUser> {
return try { Result.success(auth.createUserWithEmailAndPassword(email, pass).await().user!!) }
catch (e: Exception) { Result.failure(e) }
}
suspend fun loginWithEmail(email: String, pass: String): Result<FirebaseUser> {
return try { Result.success(auth.signInWithEmailAndPassword(email, pass).await().user!!) }
catch (e: Exception) { Result.failure(e) }
}
// FIX: reload() sincroniza el token con el servidor — sin esto isEmailVerified
// puede quedar false aunque el usuario ya verificó su email.
suspend fun reloadUser(): Result<Unit> {
return try { auth.currentUser?.reload()?.await(); Result.success(Unit) }
catch (e: Exception) { Result.failure(e) }
}
suspend fun sendEmailVerification(): Result<Unit> {
return try {
auth.currentUser?.sendEmailVerification()?.await()
?: return Result.failure(Exception("Usuario no autenticado"))
Result.success(Unit)
} catch (e: Exception) { Result.failure(e) }
}
suspend fun sendPasswordResetEmail(email: String): Result<Unit> {
return try { auth.sendPasswordResetEmail(email).await(); Result.success(Unit) }
catch (e: Exception) { Result.failure(e) }
}
suspend fun loginWithGoogle(idToken: String): Result<FirebaseUser> {
return try {
val cred = GoogleAuthProvider.getCredential(idToken, null)
Result.success(auth.signInWithCredential(cred).await().user!!)
} catch (e: Exception) { Result.failure(e) }
}
fun logout() = auth.signOut()
}
```

ui/auth/AuthViewModel.kt

```
package com.simplecallapp.ui.auth
import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
```

```
import com.simplecallapp.data.repository.AuthRepository
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.launch
sealed class AuthUiState {
object Idle : AuthUiState()
object Loading : AuthUiState()
data class Registered(val email: String) : AuthUiState()
data class LoggedIn(val uid: String) : AuthUiState()
data class NeedsVerification(val email: String) : AuthUiState()
data class Error(val message: String) : AuthUiState()
}
class AuthViewModel : ViewModel() {
private val repo = AuthRepository()
private val _state = MutableStateFlow<AuthUiState>(AuthUiState.Idle)
val state: StateFlow<AuthUiState> = _state
fun register(email: String, pass: String) = viewModelScope.launch {
_state.value = AuthUiState.Loading
val r = repo.registerWithEmail(email, pass)
if (r.isFailure) { _state.value = AuthUiState.Error(translate(r.exceptionOrNull()?.message));
return@launch }
val v = repo.sendEmailVerification()
if (v.isFailure) { repo.logout()
_state.value = AuthUiState.Error("Cuenta creada pero no se pudo enviar el email. Intenta iniciar
sesión.")
return@launch }
repo.logout()
_state.value = AuthUiState.Registered(email)
}
fun login(email: String, pass: String) = viewModelScope.launch {
_state.value = AuthUiState.Loading
val r = repo.loginWithEmail(email, pass)
if (r.isFailure) { _state.value = AuthUiState.Error(translate(r.exceptionOrNull()?.message));
return@launch }
val user = r.getOrNull()!!
repo.reloadUser() // FIX: sincronizar token antes de verificar
val isPwd = user.providerData.any { it.providerId == "password" }
if (isPwd && !user.isEmailVerified) {
repo.logout()
_state.value = AuthUiState.NeedsVerification(email)
} else {
_state.value = AuthUiState.LoggedIn(user.uid)
}
}
fun loginGoogle(idToken: String) = viewModelScope.launch {
_state.value = AuthUiState.Loading
val r = repo.loginWithGoogle(idToken)
_state.value = if (r.isSuccess) AuthUiState.LoggedIn(r.getOrNull()!!.uid)
else AuthUiState.Error(translate(r.exceptionOrNull()?.message))
}
fun resendVerification(email: String, pass: String) = viewModelScope.launch {
_state.value = AuthUiState.Loading
val r = repo.loginWithEmail(email, pass)
```

```
if (r.isFailure) { _state.value = AuthUiState.Error("Contraseña incorrecta"); return@launch }
val v = repo.sendEmailVerification()
repo.logout()
_state.value = if (v.isSuccess) AuthUiState.NeedsVerification(email)
else AuthUiState.Error("No se pudo enviar. Espera unos minutos.")
}
fun sendPasswordReset(email: String) = viewModelScope.launch {
_state.value = AuthUiState.Loading
val r = repo.sendPasswordResetEmail(email)
_state.value = if (r.isSuccess)
AuthUiState.Error("✓ Email enviado a $email. Revisa spam también.")
else AuthUiState.Error(translate(r.exceptionOrNull()?.message))
}
private fun translate(msg: String?): String {
if (msg == null) return "Error desconocido. Intenta de nuevo."
return when {
msg.contains("already in use",true) -> "Este email ya tiene cuenta. Intenta iniciar sesión."
msg.contains("badly formatted",true) -> "Formato de email inválido. Ej: nombre@dominio.com"
msg.contains("password is invalid",true)||msg.contains("invalid login",true) -> "Email o contraseña
incorrectos."
msg.contains("no user record",true)||msg.contains("user not found",true) -> "No existe cuenta con ese
email."
msg.contains("password should be",true) -> "La contraseña debe tener al menos 6 caracteres."
msg.contains("network",true)||msg.contains("timeout",true) -> "Error de conexión. Verifica WiFi o
datos."
msg.contains("too many requests",true) -> "Demasiados intentos. Espera unos minutos."
else -> msg
}
}
}
```

ui/auth/LoginActivity.kt

```
package com.simplecallapp.ui.auth
import android.content.Intent
import android.os.Bundle
import android.view.View
import android.widget.Toast
import androidx.activity.result.contract.ActivityResultContracts
import androidx.activity.viewModels
import androidx.appcompat.app.AppCompatActivity
import androidx.lifecycle.lifecycleScope
import com.google.android.gms.auth.api.signin.GoogleSignIn
import com.google.android.gms.auth.api.signin.GoogleSignInOptions
import com.google.android.gms.common.api.ApiException
import com.simplecallapp.R
import com.simplecallapp.databinding.ActivityLoginBinding
import kotlinx.coroutines.launch
class LoginActivity : AppCompatActivity() {
private lateinit var b: ActivityLoginBinding
private val vm: AuthViewModel by viewModels()
private val googleLauncher = registerForActivityResult(
ActivityResultContracts.StartActivityForResult()) { result ->
try {
val acc = GoogleSignIn.getSignedInAccountFromIntent(result.data)
```

```
.getResult(ApiException::class.java)!!
vm.loginGoogle(acc.idToken!!)
} catch (e: ApiException) {
Toast.makeText(this, "Google: ${e.message}", Toast.LENGTH_SHORT).show()
}
}
override fun onCreate(s: Bundle?) {
super.onCreate(s); b = ActivityLoginBinding.inflate(layoutInflater); setContentView(b.root)
val cu = com.google.firebase.auth.FirebaseAuth.getInstance().currentUser
if (cu != null && (cu.isEmailVerified || !cu.providerData.any { it.providerId == "password" })) {
goToApp(); return
}
showLoginForm(); setupListeners(); observeState()
}
private fun setupListeners() {
b.btnLoginEmail.setOnClickListener {
val e = b.etEmail.text?.toString()?.trim() ?: ""
val p = b.etPassword.text?.toString() ?: ""
if (e.isBlank()) { toast("Escribe tu email"); return@setOnClickListener }
if (p.isBlank()) { toast("Escribe tu contraseña"); return@setOnClickListener }
vm.login(e, p)
}
b.btnRegisterEmail.setOnClickListener {
val e = b.etEmail.text?.toString()?.trim() ?: ""
val p = b.etPassword.text?.toString() ?: ""
if (e.isBlank()) { toast("Escribe tu email"); return@setOnClickListener }
if (p.length < 6) { toast("Contraseña: mínimo 6 caracteres"); return@setOnClickListener }
vm.register(e, p)
}
b.btnGoogle.setOnClickListener {
val gso = GoogleSignInOptions.Builder(GoogleSignInOptions.DEFAULT_SIGN_IN)
.requestIdToken(getString(R.string.default_web_client_id))
.requestEmail().build()
googleLauncher.launch(GoogleSignIn.getClient(this, gso).signInIntent)
}
b.btnForgotPassword.setOnClickListener {
val e = b.etEmail.text?.toString()?.trim() ?: ""
if (e.isBlank()) { toast("Escribe tu email primero"); return@setOnClickListener }
vm.sendPasswordReset(e)
}
}
private fun observeState() {
lifecycleScope.launch {
vm.state.collect { state ->
when (state) {
is AuthUiState.Loading -> showLoading(true)
is AuthUiState.Registered -> { showLoading(false); showVerification(state.email, true) }
is AuthUiState.NeedsVerification-> { showLoading(false); showVerification(state.email, false) }
is AuthUiState.LoggedIn -> { showLoading(false); goToApp() }
is AuthUiState.Error -> { showLoading(false); toast(state.message) }
is AuthUiState.Idle -> showLoading(false)
}
}
```

```
}
}
private fun showVerification(email: String, isNew: Boolean) {
b.layoutLoginForm.visibility = View.GONE
b.layoutVerification.visibility = View.VISIBLE
b.tvVerifTitle.text = if (isNew) "Verifica tu correo" else "Email no verificado"
b.tvVerifMsg.text = "Enviamos un enlace a:\n$email\n\nAbre el email, toca el enlace y vuelve aquí."
b.btnAlreadyVerified.setOnClickListener {
val p = b.etPassword.text?.toString() ?: ""
if (p.isBlank()) { showLoginForm(); b.etEmail.setText(email)
toast("Ingresa tu contraseña para continuar"); return@setOnClickListener }
vm.login(email, p)
}
b.btnResendVerif.setOnClickListener {
val p = b.etPassword.text?.toString() ?: ""
if (p.isBlank()) { showLoginForm(); b.etEmail.setText(email)
toast("Ingresa tu contraseña para reenviar"); return@setOnClickListener }
vm.resendVerification(email, p)
}
b.btnBackToLogin.setOnClickListener { showLoginForm() }
}
private fun showLoginForm() {
b.layoutLoginForm.visibility = View.VISIBLE
b.layoutVerification.visibility = View.GONE
}
private fun showLoading(l: Boolean) {
b.progressBar.visibility = if (l) View.VISIBLE else View.GONE
b.btnLoginEmail.isEnabled = !l; b.btnRegisterEmail.isEnabled = !l; b.btnGoogle.isEnabled = !l
}
private fun toast(msg: String) = Toast.makeText(this, msg, Toast.LENGTH_LONG).show()
private fun goToApp() { startActivity(Intent(this, PermissionActivity::class.java)); finish() }
}
```

ui/auth/ChooseNumberActivity.kt

```
package com.simplecallapp.ui.auth
import android.content.Intent
import android.os.Bundle
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity
import androidx.lifecycle.lifecycleScope
import com.google.firebase.auth.FirebaseAuth
import com.google.firebase.messaging.FirebaseMessaging
import com.simplecallapp.data.repository.UserRepository
import com.simplecallapp.databinding.ActivityChooseNumberBinding
import com.simplecallapp.ui.main.MainActivity
import kotlinx.coroutines.launch
import kotlinx.coroutines.tasks.await
class ChooseNumberActivity : AppCompatActivity() {
private lateinit var b: ActivityChooseNumberBinding
private val repo = UserRepository()
override fun onCreate(s: Bundle?) {
super.onCreate(s)
b = ActivityChooseNumberBinding.inflate(layoutInflater)
```

```
setContentView(b.root)
b.btnSave.setOnClickListener {
val n = b.etNumber.text?.toString()?.trim() ?: ""
if (!n.all { it.isDigit() } || n.length < 4 || n.length > 12) {
Toast.makeText(this,"Solo dígitos, 4-12 caracteres",Toast.LENGTH_SHORT).show()
return@setOnClickListener
}
lifecycleScope.launch {
val fb = FirebaseAuth.getInstance().currentUser ?: return@launch
val tok = try { FirebaseMessaging.getInstance().token.await() } catch (_:Exception) { "" }
val ok = repo.createUser(fb.uid, fb.email?:"", fb.displayName?:fb.email?:"", n)
if (ok) {
if (tok.isNotBlank()) repo.updateFcmToken(fb.uid, tok)
startActivity(Intent(this@ChooseNumberActivity, MainActivity::class.java))
finish()
} else Toast.makeText(this@ChooseNumberActivity,
"Número ya en uso. Elige otro.",Toast.LENGTH_LONG).show()
}
}
}
}
```

12. PermissionActivity — Permisos al Inicio

ui/auth/PermissionActivity.kt

```
package com.simplecallapp.ui.auth
import android.Manifest
import android.content.Intent
import android.content.pm.PackageManager
import android.os.Build
import android.os.Bundle
import android.widget.Toast
import androidx.activity.result.contract.ActivityResultContracts
import androidx.appcompat.app.AppCompatActivity
import androidx.core.content.ContextCompat
import androidx.lifecycle.lifecycleScope
import com.google.firebase.auth.FirebaseAuth
import com.simplecallapp.data.repository.UserRepository
import com.simplecallapp.databinding.ActivityPermissionBinding
import com.simplecallapp.ui.main.MainActivity
import kotlinx.coroutines.launch
class PermissionActivity : AppCompatActivity() {
private lateinit var b: ActivityPermissionBinding
private val userRepo = UserRepository()
private val perms get() = mutableListOf(Manifest.permission.RECORD_AUDIO).also {
if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU)
it.add(Manifest.permission.POST_NOTIFICATIONS)
}.toTypedArray()
private val launcher = registerForActivityResult(
ActivityResultContracts.RequestMultiplePermissions()) { results ->
val audioOk = results[Manifest.permission.RECORD_AUDIO] == true
if (audioOk) proceed()
else { b.tvStatus.text="El micrófono es obligatorio para llamadas."
b.btnGrant.text="Intentar de nuevo" }
}
override fun onCreate(s: Bundle?) {
super.onCreate(s)
b = ActivityPermissionBinding.inflate(layoutInflater); setContentView(b.root)
if (hasAll()) { proceed(); return }
b.btnGrant.setOnClickListener { launcher.launch(perms) }
b.btnSkip.setOnClickListener {
val audio = ContextCompat.checkSelfPermission(this,Manifest.permission.RECORD_AUDIO)==PackageManager.P
ERMISSION_GRANTED
if (audio) proceed()
else Toast.makeText(this,"Sin micrófono no puedes hacer llamadas",Toast.LENGTH_LONG).show()
}
}
private fun hasAll() = perms.all {
ContextCompat.checkSelfPermission(this,it)==PackageManager.PERMISSION_GRANTED }
private fun proceed() {
lifecycleScope.launch {
val uid = FirebaseAuth.getInstance().currentUser?.uid ?: return@launch
val user = userRepo.getUser(uid)
val dest = if (user?.phoneNumber.isNullOrBlank()) ChooseNumberActivity::class.java
else MainActivity::class.java
```

```
startActivity(Intent(this@PermissionActivity, dest)); finish()
}
}
}
```

13. Repositorios de Datos

data/repository/UserRepository.kt

```
package com.simplecallapp.data.repository
import com.google.firebase.firestore.FirebaseFirestore
import com.simplecallapp.data.model.User
import kotlinx.coroutines.tasks.await
class UserRepository {
private val col = FirebaseFirestore.getInstance().collection("users")
suspend fun getUser(uid: String): User? = try {
col.document(uid).get().await().toObject(User::class.java) } catch (_:Exception) { null }
suspend fun isNumberTaken(number: String): Boolean = try {
!col.whereEqualTo("phoneNumber",number).get().await().isEmpty } catch (_:Exception) { false }
suspend fun createUser(uid:String,email:String,displayName:String,number:String): Boolean {
return try {
if (isNumberTaken(number)) return false
col.document(uid).set(User(uid,email,displayName,number)).await(); true
} catch (_:Exception) { false }
}
suspend fun updateFcmToken(uid: String, token: String) {
try { col.document(uid).update("fcmToken",token).await() } catch (_:Exception) {}
}
suspend fun getUserByNumber(number: String): User? = try {
col.whereEqualTo("phoneNumber",number).get().await()
.documents.firstOrNull()?.toObject(User::class.java) } catch (_:Exception) { null }
}
```

data/repository/ChatRepository.kt

```
package com.simplecallapp.data.repository
import com.google.firebase.firestore.FirebaseFirestore
import com.google.firebase.firestore.Query
import com.simplecallapp.data.model.Message
import kotlinx.coroutines.channels.awaitClose
import kotlinx.coroutines.flow.Flow
import kotlinx.coroutines.flow.callbackFlow
import kotlinx.coroutines.tasks.await
class ChatRepository {
private val col = FirebaseFirestore.getInstance().collection("messages")
suspend fun sendMessage(msg: Message) {
val ref = col.document()
ref.set(msg.copy(id=ref.id, timestamp=System.currentTimeMillis())).await()
}
fun listenConversation(myUid: String, otherUid: String): Flow<List<Message>> =
callbackFlow {
val merged = mutableMapOf<String,Message>()
val s1 = col.whereEqualTo("from",myUid).whereEqualTo("to",otherUid)
.orderBy("timestamp",Query.Direction.ASCENDING)
.addSnapshotListener { snap,_ ->
snap?.toObjects(Message::class.java)?.forEach { merged[it.id]=it }
trySend(merged.values.sortedBy { it.timestamp })
}
val s2 = col.whereEqualTo("from",otherUid).whereEqualTo("to",myUid)
.orderBy("timestamp",Query.Direction.ASCENDING)
```

```
.addSnapshotListener { snap,_ ->
snap?.toObjects(Message::class.java)?.forEach { merged[it.id]=it }
trySend(merged.values.sortedBy { it.timestamp })
}
awaitClose { s1.remove(); s2.remove() }
}
// FIX: sin inline prefs — el Fragment decide si llamar
suspend fun markMessagesAsRead(myUid: String, otherUid: String) {
try {
val unread = col.whereEqualTo("from",otherUid).whereEqualTo("to",myUid)
.whereEqualTo("read",false).get().await()
if (unread.isEmpty) return
val batch = FirebaseFirestore.getInstance().batch()
unread.documents.forEach { batch.update(it.reference,"read",true) }
batch.commit().await()
} catch (_:Exception) {}
}
suspend fun getLastMessages(myUid: String): List<Message> {
return try {
val sent = col.whereEqualTo("from",myUid)
.orderBy("timestamp",Query.Direction.DESCENDING).limit(200)
.get().await().toObjects(Message::class.java)
val recv = col.whereEqualTo("to",myUid)
.orderBy("timestamp",Query.Direction.DESCENDING).limit(200)
.get().await().toObjects(Message::class.java)
val byConv = mutableMapOf<String,Message>()
(sent+recv).sortedByDescending { it.timestamp }.forEach { msg ->
val other = if (msg.from==myUid) msg.to else msg.from
if (!byConv.containsKey(other)) byConv[other] = msg
}
byConv.values.sortedByDescending { it.timestamp }
} catch (_:Exception) { emptyList() }
}
}
```

data/repository/ContactRepository.kt

```
package com.simplecallapp.data.repository
import com.google.firebase.firestore.FirebaseFirestore
import com.simplecallapp.data.model.Contact
import kotlinx.coroutines.channels.awaitClose
import kotlinx.coroutines.flow.Flow; import kotlinx.coroutines.flow.callbackFlow
import kotlinx.coroutines.tasks.await
class ContactRepository {
private val col = FirebaseFirestore.getInstance().collection("contacts")
fun listenContacts(ownerUid: String): Flow<List<Contact>> = callbackFlow {
val sub = col.whereEqualTo("ownerUid",ownerUid).orderBy("name")
.addSnapshotListener { snap,_ -> trySend(snap?.toObjects(Contact::class.java)?:emptyList()) }
awaitClose { sub.remove() }
}
suspend fun addContact(c: Contact): Boolean {
return try { val ref=col.document(); col.document(ref.id).set(c.copy(id=ref.id)).await(); true }
catch (_:Exception) { false }
}
```

```
suspend fun deleteContact(id: String) {
try { col.document(id).delete().await() } catch (_:Exception) {}
}
}
```

data/repository/CallHistoryRepository.kt

```
package com.simplecallapp.data.repository
import com.google.firebase.firestore.FirebaseFirestore
import com.google.firebase.firestore.Query
import com.simplecallapp.data.model.CallHistory
import kotlinx.coroutines.tasks.await
class CallHistoryRepository {
private val col = FirebaseFirestore.getInstance().collection("call_history")
suspend fun saveCall(call: CallHistory) {
try { val ref=col.document()
col.document(ref.id).set(call.copy(id=ref.id,timestamp=System.currentTimeMillis())).await()
} catch (_:Exception) {}
}
suspend fun getHistory(myUid: String): List<CallHistory> {
return try {
val a = col.whereEqualTo("callerUid",myUid)
.orderBy("timestamp",Query.Direction.DESCENDING).limit(50).get().await().toObjects(CallHistory::class.
java)
val b = col.whereEqualTo("receiverUid",myUid)
.orderBy("timestamp",Query.Direction.DESCENDING).limit(50).get().await().toObjects(CallHistory::class.
java)
(a+b).sortedByDescending { it.timestamp }
} catch (_:Exception) { emptyList() }
}
}
```

14. WebRTCClient.kt — Corregido (sin crash de doble init)

webrtc/CallState.kt

```
package com.simplecallapp.webrtc
enum class CallState { IDLE, CALLING, RINGING, CONNECTED, ENDED, ERROR }
```

webrtc/WebRTCClient.kt

```
package com.simplecallapp.webrtc
import android.content.Context; import android.util.Log
import org.webrtc.*; import org.webrtc.audio.JavaAudioDeviceModule
class WebRTCClient(private val ctx: Context, val observer: PeerConnection.Observer) {
private val TAG = "WebRTCClient"
private val factory: PeerConnectionFactory
private var peerConnection: PeerConnection? = null
private var localAudioTrack: AudioTrack? = null
private var remoteAudioTrack: AudioTrack? = null
// Reemplazar con credenciales de app.metered.ca
private val iceServers = listOf(
PeerConnection.IceServer.builder("stun:stun.l.google.com:19302").createIceServer(),
PeerConnection.IceServer.builder("stun:stun1.l.google.com:19302").createIceServer(),
PeerConnection.IceServer.builder("turn:openrelay.metered.ca:80")
.setUsername("openrelayproject").setPassword("openrelayproject").createIceServer(),
PeerConnection.IceServer.builder("turn:openrelay.metered.ca:443")
.setUsername("openrelayproject").setPassword("openrelayproject").createIceServer(),
PeerConnection.IceServer.builder("turns:openrelay.metered.ca:443")
.setUsername("openrelayproject").setPassword("openrelayproject").createIceServer(),
)
companion object {
@Volatile private var initialized = false
fun initOnce(context: Context) {
if (!initialized) synchronized(this) {
if (!initialized) {
PeerConnectionFactory.initialize(
PeerConnectionFactory.InitializationOptions.builder(context.applicationContext)
.setEnableInternalTracer(false).createInitializationOptions())
initialized = true
}
}
}
}
init {
initOnce(ctx)
val adm = JavaAudioDeviceModule.builder(ctx)
.setUseHardwareAcousticEchoCanceler(true)
.setUseHardwareNoiseSuppressor(true).createAudioDeviceModule()
factory = PeerConnectionFactory.builder().setAudioDeviceModule(adm).createPeerConnectionFactory()
}
fun initializePeerConnection() {
val cfg = PeerConnection.RTCConfiguration(iceServers).apply {
sdpSemantics = PeerConnection.SdpSemantics.UNIFIED_PLAN
iceTransportsType = PeerConnection.IceTransportsType.ALL
bundlePolicy = PeerConnection.BundlePolicy.MAXBUNDLE
continualGatheringPolicy = PeerConnection.ContinualGatheringPolicy.GATHER_CONTINUALLY
```

```
}
peerConnection = factory.createPeerConnection(cfg, observer)
?: throw IllegalStateException("No se pudo crear PeerConnection")
}
fun addLocalAudioTrack() {
localAudioTrack = factory.createAudioTrack("LOCAL_AUDIO",
factory.createAudioSource(MediaConstraints()))
localAudioTrack?.setEnabled(true)
peerConnection?.addTrack(localAudioTrack!!, listOf("STREAM_ID"))
}
fun setRemoteAudioTrack(track: AudioTrack) {
remoteAudioTrack = track; remoteAudioTrack?.setEnabled(true)
Log.d(TAG, "Audio remoto OK")
}
fun createOffer(onSuccess: (SessionDescription) -> Unit) {
val c = MediaConstraints().apply {
mandatory.add(MediaConstraints.KeyValuePair("OfferToReceiveAudio","true"))
mandatory.add(MediaConstraints.KeyValuePair("OfferToReceiveVideo","false"))
}
peerConnection?.createOffer(sdpObs(onSuccess), c)
}
fun createAnswer(remoteSdp: String, onSuccess: (SessionDescription) -> Unit) {
peerConnection?.setRemoteDescription(object : SdpObserver {
override fun onCreateSuccess(p: SessionDescription?) {}
override fun onSetSuccess() {
val c = MediaConstraints().apply {
mandatory.add(MediaConstraints.KeyValuePair("OfferToReceiveAudio","true")) }
peerConnection?.createAnswer(sdpObs(onSuccess), c)
}
override fun onCreateFailure(e: String?) { Log.e(TAG,"setOffer: $e") }
override fun onSetFailure(e: String?) { Log.e(TAG,"setOffer: $e") }
}, SessionDescription(SessionDescription.Type.OFFER, remoteSdp))
}
fun setRemoteAnswer(sdp: String) {
peerConnection?.setRemoteDescription(object : SdpObserver {
override fun onCreateSuccess(p: SessionDescription?) {}
override fun onSetSuccess() { Log.d(TAG,"Remote answer OK") }
override fun onCreateFailure(e: String?) {}
override fun onSetFailure(e: String?) { Log.e(TAG,"answer: $e") }
}, SessionDescription(SessionDescription.Type.ANSWER, sdp))
}
fun addIceCandidate(c: IceCandidate) { peerConnection?.addIceCandidate(c) }
fun setMicMuted(muted: Boolean) { localAudioTrack?.setEnabled(!muted) }
fun disconnect() {
try { peerConnection?.close(); peerConnection=null
localAudioTrack?.dispose(); localAudioTrack=null; remoteAudioTrack=null
} catch (e: Exception) { Log.e(TAG,"disconnect: ${e.message}") }
}
private fun sdpObs(onSuccess: (SessionDescription)->Unit) = object : SdpObserver {
override fun onCreateSuccess(desc: SessionDescription) {
peerConnection?.setLocalDescription(object : SdpObserver {
override fun onCreateSuccess(p: SessionDescription?) {}
override fun onSetSuccess() { onSuccess(desc) }
```

```
override fun onCreateFailure(e: String?) {}
override fun onSetFailure(e: String?) { Log.e(TAG,"setLocal: $e") }
}, desc)
}
override fun onSetSuccess() {}
override fun onCreateFailure(e: String?) { Log.e(TAG,"createSdp: $e") }
override fun onSetFailure(e: String?) {}
}
}
```

15. WebRTCSignaling.kt — Solo ICE ADDED (sin duplicados)

webrtc/WebRTCSignaling.kt

```
package com.simplecallapp.webrtc
import com.google.firebase.firestore.DocumentChange
import com.google.firebase.firestore.FirebaseFirestore
import com.google.firebase.firestore.ListenerRegistration
import kotlinx.coroutines.tasks.await
class WebRTCSignaling {
private val calls = FirebaseFirestore.getInstance().collection("calls")
private val subs = mutableListOf<ListenerRegistration>()
suspend fun sendOffer(callId:String,callerUid:String,receiverUid:String,sdp:String) {
calls.document(callId).set(mapOf(
"callId" to callId, "callerUid" to callerUid,
"receiverUid" to receiverUid, "offer" to sdp,
"status" to "ringing", "timestamp" to System.currentTimeMillis()
)).await()
}
suspend fun sendAnswer(callId: String, sdp: String) {
calls.document(callId).update(mapOf("answer" to sdp,"status" to "connected")).await()
}
suspend fun sendIceCandidate(callId:String,isCaller:Boolean,mid:String,idx:Int,sdp:String) {
val sub = if (isCaller) "callerCandidates" else "receiverCandidates"
calls.document(callId).collection(sub).add(mapOf(
"sdpMid" to mid,"sdpMLineIndex" to idx,"sdp" to sdp)).await()
}
fun listenForAnswer(callId: String, cb: (String)->Unit) {
subs += calls.document(callId).addSnapshotListener { snap,_ ->
val a = snap?.getString("answer")
if (!a.isNullOrBlank()) cb(a)
}
}
fun listenForHangup(callId: String, cb: ()->Unit) {
subs += calls.document(callId).addSnapshotListener { snap,_ ->
if (snap?.getString("status")=="ended") cb()
}
}
// FIX: solo DocumentChange.Type.ADDED — evita ICE duplicados
fun listenForIceCandidates(callId:String, fromCaller:Boolean, cb:(String,Int,String)->Unit) {
val sub = if (fromCaller) "callerCandidates" else "receiverCandidates"
subs += calls.document(callId).collection(sub)
.addSnapshotListener { snap,err ->
if (err!=null||snap==null) return@addSnapshotListener
snap.documentChanges.filter { it.type==DocumentChange.Type.ADDED }
.forEach { ch ->
val mid = ch.document.getString("sdpMid") ?: return@forEach
val idx = ch.document.getLong("sdpMLineIndex")?.toInt() ?: return@forEach
val s = ch.document.getString("sdp") ?: return@forEach
cb(mid,idx,s)
}
}
}
```

```
suspend fun endCall(callId: String) {
try { calls.document(callId).update("status","ended").await() } catch (_:Exception) {}
}
fun clear() { subs.forEach { it.remove() }; subs.clear() }
}
```

16. CallForegroundService.kt — Ringtone Configurable

service/CallForegroundService.kt

```
package com.simplecallapp.service
import android.app.*; import android.content.*; import android.media.*; import android.os.*
import androidx.core.app.NotificationCompat
import com.simplecallapp.ui.call.CallActivity
import com.simplecallapp.ui.settings.CallSettingsFragment
import com.simplecallapp.ui.settings.NotifSettingsFragment
class CallForegroundService : Service() {
companion object {
const val CHANNEL_ID="call_service"; const val NOTIF_ID=1001
const val EXTRA_REMOTE_NAME="remote_name"; const val ACTION_STOP="STOP_RING"
fun stopRingtone(ctx: Context) {
ctx.startService(Intent(ctx,CallForegroundService::class.java).apply{ action=ACTION_STOP })
}
}
private var mp: MediaPlayer? = null
private var vib: Vibrator? = null
override fun onStartCommand(i: Intent?, flags: Int, id: Int): Int {
if (i?.action==ACTION_STOP) { stopAll(); stopForeground(STOP_FOREGROUND_DETACH); return
START_NOT_STICKY }
val name = i?.getStringExtra(EXTRA_REMOTE_NAME)?:"Llamada"
mkChannel()
val pi = PendingIntent.getActivity(this,0,
Intent(this,CallActivity::class.java).apply{flags=Intent.FLAG_ACTIVITY_SINGLE_TOP},
PendingIntent.FLAG_UPDATE_CURRENT or PendingIntent.FLAG_IMMUTABLE)
startForeground(NOTIF_ID, NotificationCompat.Builder(this,CHANNEL_ID)
.setSmallIcon(android.R.drawable.ic_menu_call)
.setContentTitle("Llamada en curso").setContentText(name)
.setOngoing(true).setContentIntent(pi).build())
if (NotifSettingsFragment.isSoundEnabled(this)) startRing()
if (NotifSettingsFragment.isVibrateEnabled(this)) startVib()
return START_NOT_STICKY
}
private fun startRing() {
try {
mp = MediaPlayer().apply {
setAudioAttributes(AudioAttributes.Builder()
.setUsage(AudioAttributes.USAGE_NOTIFICATION_RINGTONE)
.setContentType(AudioAttributes.CONTENT_TYPE_SONIFICATION).build())
// Usa el tono elegido por el usuario o el predeterminado
setDataSource(this@CallForegroundService,
CallSettingsFragment.getCallRingtoneUri(this@CallForegroundService))
isLooping=true; prepare(); start()
}
} catch (e: Exception) { e.printStackTrace() }
}
private fun startVib() {
vib = if (Build.VERSION.SDK_INT>=Build.VERSION_CODES.S)
(getSystemService(VIBRATOR_MANAGER_SERVICE) as VibratorManager).defaultVibrator
else @Suppress("DEPRECATION") getSystemService(VIBRATOR_SERVICE) as Vibrator
val pat = longArrayOf(0,1000,1000)
```

```
if (Build.VERSION.SDK_INT>=Build.VERSION_CODES.O)
vib?.vibrate(VibrationEffect.createWaveform(pat,0))
else @Suppress("DEPRECATION") vib?.vibrate(pat,0)
}
private fun stopAll() {
mp?.let { if(it.isPlaying) it.stop(); it.release() }; mp=null; vib?.cancel(); vib=null
}
override fun onDestroy() { super.onDestroy(); stopAll() }
override fun onBind(i: Intent?): IBinder? = null
private fun mkChannel() {
if (Build.VERSION.SDK_INT>=Build.VERSION_CODES.O)
(getSystemService(NOTIFICATION_SERVICE) as NotificationManager)
.createNotificationChannel(NotificationChannel(CHANNEL_ID,"Llamada
activa",NotificationManager.IMPORTANCE_LOW))
}
}
```

17. CallActivity.kt — cleanupScope y onTrack() Corregidos

ui/call/CallActivity.kt

```
package com.simplecallapp.ui.call
import android.content.Intent; import android.media.AudioManager
import android.os.Bundle; import android.os.Handler; import android.os.Looper
import android.util.Log
import androidx.appcompat.app.AppCompatActivity
import androidx.lifecycle.lifecycleScope
import com.google.firebase.auth.FirebaseAuth
import com.simplecallapp.R
import com.simplecallapp.data.model.CallHistory; import com.simplecallapp.data.model.CallType
import com.simplecallapp.data.repository.CallHistoryRepository
import com.simplecallapp.databinding.ActivityCallBinding
import com.simplecallapp.service.CallForegroundService
import com.simplecallapp.webrtc.CallState; import com.simplecallapp.webrtc.WebRTCClient; import
com.simplecallapp.webrtc.WebRTCSignaling
import kotlinx.coroutines.CoroutineScope; import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.SupervisorJob; import kotlinx.coroutines.cancel; import
kotlinx.coroutines.launch
import org.webrtc.*
class CallActivity : AppCompatActivity() {
private lateinit var b: ActivityCallBinding
private lateinit var rtc: WebRTCClient
private val sig = WebRTCSignaling()
private val hist = CallHistoryRepository()
// FIX: scope independiente para guardar historial aunque la activity muera
private val cleanupScope = CoroutineScope(Dispatchers.IO + SupervisorJob())
private var callId=""; private var isCaller=false; private var isMuted=false
private var callState=CallState.IDLE; private var callSecs=0
private var remoteUid=""; private var remoteName=""; private var callType=CallType.OUTGOING
private val timer = Handler(Looper.getMainLooper())
private val myUid get() = FirebaseAuth.getInstance().currentUser?.uid?:""
override fun onCreate(s: Bundle?) {
super.onCreate(s)
b = ActivityCallBinding.inflate(layoutInflater); setContentView(b.root)
callId = intent.getStringExtra("CALL_ID")?:""
isCaller = intent.getBooleanExtra("IS_CALLER",true)
remoteUid = intent.getStringExtra("REMOTE_UID")?:""
remoteName = intent.getStringExtra("REMOTE_NAME")?:"Desconocido"
callType = if (isCaller) CallType.OUTGOING else CallType.INCOMING
b.tvRemoteName.text = remoteName
b.tvAvatar.text = remoteName.firstOrNull()?.uppercaseChar()?.toString()?:"?"
setupAudio(); setupWebRTC(); setupButtons()
startService(Intent(this,CallForegroundService::class.java).apply {
putExtra(CallForegroundService.EXTRA_REMOTE_NAME,remoteName) })
if (isCaller) callerFlow() else receiverFlow()
}
private fun setupAudio() {
(getSystemService(AUDIO_SERVICE) as AudioManager).apply {
mode=AudioManager.MODE_IN_COMMUNICATION; isSpeakerphoneOn=false }
}
private fun setupWebRTC() {
```

```
rtc = WebRTCClient(this, object : PeerConnection.Observer {
// FIX: onTrack captura el audio remoto con UNIFIED_PLAN
override fun onTrack(t: RtpTransceiver?) {
t?.receiver?.track()?.let { if (it is AudioTrack) rtc.setRemoteAudioTrack(it) }
}
override fun onIceCandidate(c: IceCandidate) {
lifecycleScope.launch { sig.sendIceCandidate(callId,isCaller,c.sdpMid,c.sdpMLineIndex,c.sdp) }
}
override fun onConnectionChange(state: PeerConnection.PeerConnectionState?) {
runOnUiThread { when(state) {
PeerConnection.PeerConnectionState.CONNECTED -> {
callState=CallState.CONNECTED
b.tvCallState.text="En llamada"
b.tvCallState.setTextColor(getColor(R.color.green_500))
startTimer()
CallForegroundService.stopRingtone(this@CallActivity)
}
PeerConnection.PeerConnectionState.DISCONNECTED,
PeerConnection.PeerConnectionState.FAILED ->
if (callState!=CallState.ENDED) hangUp()
else -> {}
}}
}
override fun onSignalingChange(p: PeerConnection.SignalingState?) {}
override fun onIceConnectionChange(p: PeerConnection.IceConnectionState?) {}
override fun onIceConnectionReceivingChange(p: Boolean) {}
override fun onIceGatheringChange(p: PeerConnection.IceGatheringState?) {}
override fun onIceCandidatesRemoved(p: Array<out IceCandidate>?) {}
override fun onAddStream(p: MediaStream?) {}
override fun onRemoveStream(p: MediaStream?) {}
override fun onDataChannel(p: DataChannel?) {}
override fun onRenegotiationNeeded() {}
})
rtc.initializePeerConnection(); rtc.addLocalAudioTrack()
}
private fun callerFlow() {
callState=CallState.CALLING; b.tvCallState.text="Llamando..."
rtc.createOffer { sdp ->
lifecycleScope.launch {
sig.sendOffer(callId,myUid,remoteUid,sdp.description)
sig.listenForAnswer(callId) { rtc.setRemoteAnswer(it) }
sig.listenForIceCandidates(callId,false) { m,i,s -> rtc.addIceCandidate(IceCandidate(m,i,s)) }
sig.listenForHangup(callId) { runOnUiThread { if (callState!=CallState.ENDED) hangUp() } }
}
}
}
private fun receiverFlow() {
callState=CallState.RINGING; b.tvCallState.text="Llamada entrante..."
rtc.createAnswer(intent.getStringExtra("REMOTE_SDP")?:"") { sdp ->
lifecycleScope.launch {
sig.sendAnswer(callId,sdp.description)
sig.listenForIceCandidates(callId,true) { m,i,s -> rtc.addIceCandidate(IceCandidate(m,i,s)) }
sig.listenForHangup(callId) { runOnUiThread { if (callState!=CallState.ENDED) hangUp() } }
```

```
}
}
}
private fun startTimer() {
timer.post(object:Runnable{
override fun run() {
callSecs++; b.tvDuration.text="%02d:%02d".format(callSecs/60,callSecs%60)
timer.postDelayed(this,1000)
}
})
}
private fun setupButtons() {
b.btnHangup.setOnClickListener { hangUp() }
b.btnMute.setOnClickListener {
isMuted=!isMuted; rtc.setMicMuted(isMuted)
b.tvMute.text=if(isMuted)"Activar mic" else "Silenciar"
}
b.btnSpeaker.setOnClickListener {
val am=getSystemService(AUDIO_SERVICE) as AudioManager
am.isSpeakerphoneOn=!am.isSpeakerphoneOn
b.tvSpeaker.text=if(am.isSpeakerphoneOn)"Auricular" else "Altavoz"
}
}
// FIX: cleanupScope para guardar historial aunque la activity ya esté destruida
private fun hangUp() {
if (callState==CallState.ENDED) return
callState=CallState.ENDED
timer.removeCallbacksAndMessages(null)
cleanupScope.launch {
try {
sig.endCall(callId)
hist.saveCall(CallHistory(
callerUid=if(isCaller) myUid else remoteUid,
receiverUid=if(isCaller) remoteUid else myUid,
receiverNumber=remoteName, type=callType.name, durationSeconds=callSecs))
} catch (e:Exception) { Log.e("CallActivity","hangUp: ${e.message}") }
}
sig.clear(); rtc.disconnect()
(getSystemService(AUDIO_SERVICE) as AudioManager).mode=AudioManager.MODE_NORMAL
CallForegroundService.stopRingtone(this)
stopService(Intent(this,CallForegroundService::class.java))
finish()
}
override fun onDestroy() { super.onDestroy(); hangUp(); cleanupScope.cancel() }
}
```

18. MyFirebaseMessagingService.kt

service/MyFirebaseMessagingService.kt

```
package com.simplecallapp.service
import android.app.*; import android.content.*; import android.os.Build
import androidx.core.app.NotificationCompat
import com.google.firebase.auth.FirebaseAuth
import com.google.firebase.messaging.FirebaseMessagingService
import com.google.firebase.messaging.RemoteMessage
import com.simplecallapp.data.repository.UserRepository
import com.simplecallapp.ui.call.CallActivity
import com.simplecallapp.ui.settings.NotifSettingsFragment
import kotlinx.coroutines.CoroutineScope; import kotlinx.coroutines.Dispatchers; import
kotlinx.coroutines.launch
class MyFirebaseMessagingService : FirebaseMessagingService() {
override fun onMessageReceived(msg: RemoteMessage) {
val d = msg.data
when (d["type"]) {
"incoming_call" -> {
if (!NotifSettingsFragment.areCallsEnabled(this)) return
handleCall(d["callId"]?:return, d["callerUid"]?:return,
d["callerNumber"]?:"Desconocido", d["offer"]?:return)
}
"message" -> {
if (!NotifSettingsFragment.areMessagesEnabled(this)) return
showMsg(d["fromUid"]?:"", d["fromNumber"]?:return, d["text"]?:return)
}
}
}
override fun onNewToken(token: String) {
val uid = FirebaseAuth.getInstance().currentUser?.uid ?: return
CoroutineScope(Dispatchers.IO).launch { UserRepository().updateFcmToken(uid,token) }
}
private fun handleCall(callId:String,callerUid:String,callerNum:String,offer:String) {
mkChannels()
val pi = PendingIntent.getActivity(this,0,
Intent(this,CallActivity::class.java).apply {
flags=Intent.FLAG_ACTIVITY_NEW_TASK or Intent.FLAG_ACTIVITY_CLEAR_TOP
putExtra("CALL_ID",callId); putExtra("IS_CALLER",false)
putExtra("REMOTE_UID",callerUid); putExtra("REMOTE_NAME",callerNum)
putExtra("REMOTE_SDP",offer) },
PendingIntent.FLAG_UPDATE_CURRENT or PendingIntent.FLAG_IMMUTABLE)
(getSystemService(NOTIFICATION_SERVICE) as NotificationManager)
.notify(callId.hashCode(), NotificationCompat.Builder(this,"calls")
.setSmallIcon(android.R.drawable.ic_menu_call)
.setContentTitle("Llamada entrante").setContentText("De: $callerNum")
.setPriority(NotificationCompat.PRIORITY_MAX)
.setCategory(NotificationCompat.CATEGORY_CALL)
.setFullScreenIntent(pi,true).setAutoCancel(true).setOngoing(true).build())
val svc = Intent(this,CallForegroundService::class.java).apply {
putExtra(CallForegroundService.EXTRA_REMOTE_NAME,"De: $callerNum") }
if (Build.VERSION.SDK_INT>=Build.VERSION_CODES.O) startForegroundService(svc)
else startService(svc)
```

```
}
private fun showMsg(fromUid:String, fromNum:String, text:String) {
// Verificar si el contacto está silenciado
val muted = getSharedPreferences("muted_contacts",Context.MODE_PRIVATE)
.getBoolean("muted_$fromUid",false)
if (muted) return
mkChannels()
(getSystemService(NOTIFICATION_SERVICE) as NotificationManager)
.notify(fromNum.hashCode(), NotificationCompat.Builder(this,"messages")
.setSmallIcon(android.R.drawable.ic_dialog_email)
.setContentTitle("Mensaje de $fromNum").setContentText(text)
.setPriority(NotificationCompat.PRIORITY_DEFAULT).setAutoCancel(true).build())
}
private fun mkChannels() {
if (Build.VERSION.SDK_INT>=Build.VERSION_CODES.O) {
val nm = getSystemService(NOTIFICATION_SERVICE) as NotificationManager
nm.createNotificationChannel(NotificationChannel("calls","Llamadas",NotificationManager.IMPORTANCE_HIG
H).apply{
enableVibration(true); vibrationPattern=longArrayOf(0,500,250,500) })
nm.createNotificationChannel(NotificationChannel("messages","Mensajes",NotificationManager.IMPORTANCE_
DEFAULT))
}
}
}
```

19. Adapters — Corregidos

ui/chat/MessageAdapter.kt — con tvTick corregido

```
package com.simplecallapp.ui.chat
import android.view.*; import android.widget.TextView
import androidx.recyclerview.widget.DiffUtil
import androidx.recyclerview.widget.ListAdapter
import androidx.recyclerview.widget.RecyclerView
import com.simplecallapp.R; import com.simplecallapp.data.model.Message
import java.text.SimpleDateFormat; import java.util.*
class MessageAdapter(private val myUid: String) :
ListAdapter<Message,RecyclerView.ViewHolder>(
object:DiffUtil.ItemCallback<Message>(){
override fun areItemsTheSame(a:Message,b:Message)=a.id==b.id
override fun areContentsTheSame(a:Message,b:Message)=a==b}) {
companion object { const val SENT=0; const val RECV=1 }
override fun getItemViewType(p:Int)=if(getItem(p).from==myUid) SENT else RECV
override fun onCreateViewHolder(parent:ViewGroup,type:Int): RecyclerView.ViewHolder {
val inf=LayoutInflater.from(parent.context)
return if(type==SENT) SentVH(inf.inflate(R.layout.item_message_sent,parent,false))
else RecvVH(inf.inflate(R.layout.item_message_received,parent,false))
}
override fun onBindViewHolder(holder:RecyclerView.ViewHolder,pos:Int) {
val msg=getItem(pos)
val time=SimpleDateFormat("HH:mm",Locale.getDefault()).format(Date(msg.timestamp))
when(holder) {
is SentVH -> {
holder.tvText.text=msg.text; holder.tvTime.text=time
// FIX: bind tvTick — sin esto crashea con NPE
when {
msg.read -> { holder.tvTick.text="✓✓"
holder.tvTick.setTextColor(holder.itemView.context.getColor(R.color.tick_read)) }
msg.delivered -> { holder.tvTick.text="✓"
holder.tvTick.setTextColor(holder.itemView.context.getColor(R.color.tick_delivered)) }
else -> { holder.tvTick.text="■"
holder.tvTick.setTextColor(holder.itemView.context.getColor(R.color.tick_delivered)) }
}
}
is RecvVH -> { holder.tvText.text=msg.text; holder.tvTime.text=time }
}
}
// FIX: tvTick declarado aquí (faltaba en versión anterior)
class SentVH(v:View):RecyclerView.ViewHolder(v){
val tvText:TextView=v.findViewById(R.id.tvText)
val tvTime:TextView=v.findViewById(R.id.tvTime)
val tvTick:TextView=v.findViewById(R.id.tvTick) }
class RecvVH(v:View):RecyclerView.ViewHolder(v){
val tvText:TextView=v.findViewById(R.id.tvText)
val tvTime:TextView=v.findViewById(R.id.tvTime) }
}
```

ui/contacts/ContactsAdapter.kt

```
package com.simplecallapp.ui.contacts
```

```
import android.view.*; import android.widget.TextView
import androidx.recyclerview.widget.DiffUtil
import androidx.recyclerview.widget.ListAdapter
import androidx.recyclerview.widget.RecyclerView
import com.simplecallapp.R; import com.simplecallapp.data.model.Contact
class ContactsAdapter(private val onCall:(Contact)->Unit, private val onMessage:(Contact)->Unit) :
ListAdapter<Contact,ContactsAdapter.VH>(
object:DiffUtil.ItemCallback<Contact>(){
override fun areItemsTheSame(a:Contact,b:Contact)=a.id==b.id
override fun areContentsTheSame(a:Contact,b:Contact)=a==b}) {
inner class VH(v:View):RecyclerView.ViewHolder(v) {
val tvName:TextView=v.findViewById(R.id.tvName)
val tvNumber:TextView=v.findViewById(R.id.tvNumber)
val btnCall:View=v.findViewById(R.id.btnCall)
val btnMsg:View=v.findViewById(R.id.btnMessage)
}
override fun onCreateViewHolder(p:ViewGroup,t:Int)=
VH(LayoutInflater.from(p.context).inflate(R.layout.item_contact,p,false))
override fun onBindViewHolder(h:VH,pos:Int) {
val c=getItem(pos)
h.tvName.text=c.name; h.tvNumber.text=c.number
h.btnCall.setOnClickListener{onCall(c)}; h.btnMsg.setOnClickListener{onMessage(c)}
}
}
```

ui/calls/CallHistoryAdapter.kt

```
package com.simplecallapp.ui.calls
import android.view.*; import android.widget.TextView
import androidx.recyclerview.widget.DiffUtil
import androidx.recyclerview.widget.ListAdapter
import androidx.recyclerview.widget.RecyclerView
import com.simplecallapp.R
import com.simplecallapp.data.model.CallHistory; import com.simplecallapp.data.model.CallType
import java.text.SimpleDateFormat; import java.util.*
class CallHistoryAdapter(private val myUid:String, private val onCall:(String)->Unit) :
ListAdapter<CallHistory,CallHistoryAdapter.VH>(
object:DiffUtil.ItemCallback<CallHistory>(){
override fun areItemsTheSame(a:CallHistory,b:CallHistory)=a.id==b.id
override fun areContentsTheSame(a:CallHistory,b:CallHistory)=a==b}) {
inner class VH(v:View):RecyclerView.ViewHolder(v) {
val tvName:TextView=v.findViewById(R.id.tvName)
val tvType:TextView=v.findViewById(R.id.tvType)
val tvDuration:TextView=v.findViewById(R.id.tvDuration)
val tvDate:TextView=v.findViewById(R.id.tvDate)
}
override fun onCreateViewHolder(p:ViewGroup,t:Int)=
VH(LayoutInflater.from(p.context).inflate(R.layout.item_call_history,p,false))
override fun onBindViewHolder(h:VH,pos:Int) {
val c=getItem(pos)
val other=if(c.callerUid==myUid) c.receiverNumber else c.callerNumber
h.tvName.text=other.ifBlank{"Desconocido"}
h.tvDate.text=SimpleDateFormat("dd/MM HH:mm",Locale.getDefault()).format(Date(c.timestamp))
h.tvDuration.text="%02d:%02d".format(c.durationSeconds/60,c.durationSeconds%60)
```

```
h.tvType.text=when(c.type){
CallType.OUTGOING.name->"Saliente"; CallType.INCOMING.name->"Entrante"; else->"Perdida" }
h.itemView.setOnClickListener{onCall(other)}
}
}
```

20. Fragments — Sección Llamadas

ui/calls/CallsFragment.kt — con botón de configuración

```
package com.simplecallapp.ui.calls
import android.os.Bundle; import android.view.*
import androidx.fragment.app.Fragment
import androidx.navigation.fragment.findNavController
import androidx.viewpager2.adapter.FragmentStateAdapter
import com.google.android.material.tabs.TabLayoutMediator
import com.simplecallapp.R; import com.simplecallapp.databinding.FragmentCallsBinding
class CallsFragment : Fragment() {
private var _b: FragmentCallsBinding? = null
private val b get() = _b!!
override fun onCreateView(i:LayoutInflater,c:ViewGroup?,s:Bundle?): View {
_b=FragmentCallsBinding.inflate(i,c,false); return b.root
}
override fun onViewCreated(v:View,s:Bundle?) {
super.onViewCreated(v,s)
b.viewPager.adapter=object:FragmentStateAdapter(this){
override fun getItemCount()=2
override fun createFragment(p:Int)=if(p==0) DialFragment() else CallHistoryFragment()
}
TabLayoutMediator(b.tabLayout,b.viewPager){ tab,pos->
tab.text=if(pos==0)"Teclado" else "Recientes"
}.attach()
// Botón de configuración de llamadas
b.btnCallSettings.setOnClickListener {
findNavController().navigate(R.id.action_calls_to_call_settings)
}
}
override fun onDestroyView() { super.onDestroyView(); _b=null }
}
```

ui/calls/DialFragment.kt — con permiso RECORD_AUDIO

```
package com.simplecallapp.ui.calls
import android.Manifest; import android.content.Intent
import android.content.pm.PackageManager
import android.os.Bundle; import android.view.*; import android.widget.Toast
import androidx.activity.result.contract.ActivityResultContracts
import androidx.core.content.ContextCompat
import androidx.fragment.app.Fragment; import androidx.lifecycle.lifecycleScope
import com.google.firebase.auth.FirebaseAuth
import com.simplecallapp.data.repository.UserRepository
import com.simplecallapp.databinding.FragmentDialBinding
import com.simplecallapp.ui.call.CallActivity
import kotlinx.coroutines.launch; import java.util.UUID
class DialFragment : Fragment() {
private var _b: FragmentDialBinding? = null
private val b get() = _b!!
private val userRepo=UserRepository()
private val dialNum=StringBuilder()
private var pendingNum=""
private val audioLauncher=registerForActivityResult(
```

```
ActivityResultContracts.RequestPermission()){ granted->
if(granted) doCall(pendingNum)
else Toast.makeText(requireContext(),"Permiso de micrófono requerido",Toast.LENGTH_LONG).show()
}
override fun onCreateView(i:LayoutInflater,c:ViewGroup?,s:Bundle?): View {
_b=FragmentDialBinding.inflate(i,c,false); return b.root
}
override fun onViewCreated(v:View,s:Bundle?) {
super.onViewCreated(v,s)
setupPad()
b.btnCall.setOnClickListener {
val n=dialNum.toString()
if(n.isBlank()){Toast.makeText(requireContext(),"Ingresa un número",Toast.LENGTH_SHORT).show();
return@setOnClickListener}
pendingNum=n
if(ContextCompat.checkSelfPermission(requireContext(),Manifest.permission.RECORD_AUDIO)==PackageManage
r.PERMISSION_GRANTED)
doCall(n) else audioLauncher.launch(Manifest.permission.RECORD_AUDIO)
}
}
private fun doCall(number:String) {
lifecycleScope.launch {
try {
val remote=userRepo.getUserByNumber(number)
if(remote==null){Toast.makeText(requireContext(),"Usuario no encontrado",Toast.LENGTH_SHORT).show();
return@launch}
val myUid=FirebaseAuth.getInstance().currentUser?.uid?:return@launch
if(remote.uid==myUid){Toast.makeText(requireContext(),"No puedes llamarte a ti
mismo",Toast.LENGTH_SHORT).show(); return@launch}
startActivity(Intent(requireContext(),CallActivity::class.java).apply{
putExtra("CALL_ID",UUID.randomUUID().toString())
putExtra("IS_CALLER",true)
putExtra("REMOTE_UID",remote.uid)
putExtra("REMOTE_NAME",remote.phoneNumber)})
} catch(e:Exception){Toast.makeText(requireContext(),"Error: ${e.message}",Toast.LENGTH_SHORT).show()}
}
}
private fun setupPad() {
val m=mapOf(b.btn0 to "0",b.btn1 to "1",b.btn2 to "2",b.btn3 to "3",b.btn4 to "4",
b.btn5 to "5",b.btn6 to "6",b.btn7 to "7",b.btn8 to "8",b.btn9 to "9",b.btnStar to "*")
m.forEach{(btn,d)->btn.setOnClickListener{if(dialNum.length<12){dialNum.append(d);
b.tvDialNumber.text=dialNum.toString()}}}
b.btnBackspace.setOnClickListener{if(dialNum.isNotEmpty()){dialNum.deleteCharAt(dialNum.length-1);
b.tvDialNumber.text=dialNum.toString()}}
b.btnBackspace.setOnLongClickListener{dialNum.clear(); b.tvDialNumber.text=""; true}
}
override fun onDestroyView() { super.onDestroyView(); _b=null }
}
```

ui/calls/CallHistoryFragment.kt — con onResume()

```
package com.simplecallapp.ui.calls
import android.content.Intent; import android.os.Bundle; import android.view.*
import androidx.fragment.app.Fragment; import androidx.lifecycle.lifecycleScope
import androidx.recyclerview.widget.LinearLayoutManager
import com.google.firebase.auth.FirebaseAuth
```

```
import com.simplecallapp.data.repository.CallHistoryRepository
import com.simplecallapp.data.repository.UserRepository
import com.simplecallapp.databinding.FragmentCallHistoryBinding
import com.simplecallapp.ui.call.CallActivity
import kotlinx.coroutines.launch; import java.util.UUID
class CallHistoryFragment : Fragment() {
private var _b: FragmentCallHistoryBinding? = null
private val b get() = _b!!
private val histRepo=CallHistoryRepository(); private val userRepo=UserRepository()
private lateinit var adapter: CallHistoryAdapter
private val myUid get()=FirebaseAuth.getInstance().currentUser?.uid?:""
override fun onCreateView(i:LayoutInflater,c:ViewGroup?,s:Bundle?): View {
_b=FragmentCallHistoryBinding.inflate(i,c,false); return b.root
}
override fun onViewCreated(v:View,s:Bundle?) {
super.onViewCreated(v,s)
adapter=CallHistoryAdapter(myUid){num->callBack(num)}
b.recycler.layoutManager=LinearLayoutManager(requireContext())
b.recycler.adapter=adapter; loadHistory()
}
// FIX: onResume recarga el historial al volver a la pestaña
override fun onResume() { super.onResume(); loadHistory() }
private fun loadHistory() {
lifecycleScope.launch {
val h=histRepo.getHistory(myUid); adapter.submitList(h)
b.tvEmpty.visibility=if(h.isEmpty()) View.VISIBLE else View.GONE
}
}
private fun callBack(number:String) {
lifecycleScope.launch {
val r=userRepo.getUserByNumber(number)?:return@launch
startActivity(Intent(requireContext(),CallActivity::class.java).apply{
putExtra("CALL_ID",UUID.randomUUID().toString())
putExtra("IS_CALLER",true); putExtra("REMOTE_UID",r.uid); putExtra("REMOTE_NAME",r.phoneNumber)})
}
}
override fun onDestroyView() { super.onDestroyView(); _b=null }
}
```

21. Fragments — Sección Mensajes

ui/chat/ConversationsFragment.kt — con onResume() y clave canónica

```
package com.simplecallapp.ui.chat
import android.app.AlertDialog; import android.os.Bundle; import android.view.*
import android.widget.*; import androidx.core.os.bundleOf; import androidx.fragment.app.Fragment
import androidx.lifecycle.lifecycleScope; import androidx.navigation.fragment.findNavController
import androidx.recyclerview.widget.DiffUtil; import androidx.recyclerview.widget.LinearLayoutManager
import androidx.recyclerview.widget.ListAdapter; import androidx.recyclerview.widget.RecyclerView
import com.google.firebase.auth.FirebaseAuth
import com.simplecallapp.R; import com.simplecallapp.data.model.Message
import com.simplecallapp.data.repository.ChatRepository; import
com.simplecallapp.data.repository.UserRepository
import com.simplecallapp.databinding.FragmentConversationsBinding
import kotlinx.coroutines.launch; import java.text.SimpleDateFormat; import java.util.*
class ConversationsFragment : Fragment() {
private var _b: FragmentConversationsBinding? = null
private val b get() = _b!!
private val chatRepo=ChatRepository(); private val userRepo=UserRepository()
private val myUid get()=FirebaseAuth.getInstance().currentUser?.uid?:""
private lateinit var adapter: ConvAdapter
override fun onCreateView(i:LayoutInflater,c:ViewGroup?,s:Bundle?): View {
_b=FragmentConversationsBinding.inflate(i,c,false); return b.root
}
override fun onViewCreated(v:View,s:Bundle?) {
super.onViewCreated(v,s)
adapter=ConvAdapter{ msg->
val otherUid=if(msg.from==myUid) msg.to else msg.from
val otherNum=if(msg.from==myUid) msg.toNumber else msg.fromNumber
findNavController().navigate(R.id.action_conversations_to_chat,
bundleOf("otherUid" to otherUid,"otherNumber" to otherNum))
}
b.recycler.layoutManager=LinearLayoutManager(requireContext())
b.recycler.adapter=adapter; loadConversations()
b.fabNewChat.setOnClickListener{ showNewChatDialog() }
}
// FIX: refrescar lista al volver a esta pantalla
override fun onResume() { super.onResume(); loadConversations() }
private fun loadConversations() {
lifecycleScope.launch {
val msgs=chatRepo.getLastMessages(myUid); adapter.submitList(msgs)
b.tvEmpty.visibility=if(msgs.isEmpty()) View.VISIBLE else View.GONE
}
}
private fun showNewChatDialog() {
val input=EditText(requireContext()).apply{hint="Número del destinatario";
inputType=android.text.InputType.TYPE_CLASS_NUMBER}
AlertDialog.Builder(requireContext()).setTitle("Nueva conversación").setView(input)
.setPositiveButton("Ir"){ _,_->
val n=input.text.toString().trim(); if(n.isBlank()) return@setPositiveButton
lifecycleScope.launch {
val r=userRepo.getUserByNumber(n)
```

```
if(r==null){Toast.makeText(requireContext(),"Usuario no encontrado",Toast.LENGTH_SHORT).show();
return@launch}
findNavController().navigate(R.id.action_conversations_to_chat,
bundleOf("otherUid" to r.uid,"otherNumber" to r.phoneNumber))
}
}.setNegativeButton("Cancelar",null).show()
}
class ConvAdapter(private val onClick:(Message)->Unit):
ListAdapter<Message,ConvAdapter.VH>(object:DiffUtil.ItemCallback<Message>(){
// FIX: clave canónica — evita colisiones de string
override fun areItemsTheSame(a:Message,b:Message): Boolean {
val kA=minOf(a.from,a.to)+"_"+maxOf(a.from,a.to)
val kB=minOf(b.from,b.to)+"_"+maxOf(b.from,b.to)
return kA==kB
}
override fun areContentsTheSame(a:Message,b:Message)=a==b}) {
inner class VH(v:View):RecyclerView.ViewHolder(v){
val tvName:TextView=v.findViewById(R.id.tvName)
val tvLast:TextView=v.findViewById(R.id.tvLastMessage)
val tvTime:TextView=v.findViewById(R.id.tvTime)
}
override fun onCreateViewHolder(p:ViewGroup,t:Int)=VH(
LayoutInflater.from(p.context).inflate(R.layout.item_conversation,p,false))
override fun onBindViewHolder(h:VH,pos:Int) {
val m=getItem(pos)
h.tvName.text=m.fromNumber.ifBlank{m.toNumber}; h.tvLast.text=m.text
h.tvTime.text=SimpleDateFormat("HH:mm",Locale.getDefault()).format(Date(m.timestamp))
h.itemView.setOnClickListener{onClick(m)}
}
}
override fun onDestroyView() { super.onDestroyView(); _b=null }
}
```

ui/chat/ChatFragment.kt — sin navArgs, import Context incluido

```
package com.simplecallapp.ui.chat
import android.content.Context // FIX: import faltante
import android.os.Bundle; import android.view.*; import android.widget.Toast
import androidx.core.view.MenuProvider
import androidx.fragment.app.Fragment; import androidx.lifecycle.lifecycleScope
import androidx.recyclerview.widget.LinearLayoutManager
import com.google.firebase.auth.FirebaseAuth
import com.simplecallapp.R; import com.simplecallapp.data.model.Message
import com.simplecallapp.data.repository.ChatRepository; import
com.simplecallapp.data.repository.UserRepository
import com.simplecallapp.databinding.FragmentChatBinding
import kotlinx.coroutines.flow.launchIn; import kotlinx.coroutines.flow.onEach
import kotlinx.coroutines.launch
// FIX: NO incluir 'import androidx.navigation.fragment.navArgs' — no hay SafeArgs plugin
class ChatFragment : Fragment() {
private var _b: FragmentChatBinding? = null
private val b get() = _b!!
private val chatRepo=ChatRepository(); private val userRepo=UserRepository()
private lateinit var adapter: MessageAdapter
private var otherUid=""; private var otherNumber=""; private var myNumber=""
```

```
private val myUid get()=FirebaseAuth.getInstance().currentUser?.uid?:""
override fun onCreateView(i:LayoutInflater,c:ViewGroup?,s:Bundle?): View {
_b=FragmentChatBinding.inflate(i,c,false); return b.root
}
override fun onViewCreated(v:View,s:Bundle?) {
super.onViewCreated(v,s)
otherUid=arguments?.getString("otherUid")?:""
otherNumber=arguments?.getString("otherNumber")?:""
b.tvTitle.text=otherNumber
adapter=MessageAdapter(myUid)
b.recycler.layoutManager=LinearLayoutManager(requireContext()).apply{stackFromEnd=true}
b.recycler.adapter=adapter
// FIX: deshabilitar Enviar hasta cargar myNumber
b.btnSend.isEnabled=false
lifecycleScope.launch {
myNumber=userRepo.getUser(myUid)?.phoneNumber?:""
b.btnSend.isEnabled=true
}
val prefs=requireContext().getSharedPreferences("msg_prefs",Context.MODE_PRIVATE)
if(prefs.getBoolean("send_read_receipts",true))
lifecycleScope.launch { chatRepo.markMessagesAsRead(myUid,otherUid) }
chatRepo.listenConversation(myUid,otherUid).onEach{ msgs->
adapter.submitList(msgs)
if(msgs.isNotEmpty()) b.recycler.scrollToPosition(msgs.size-1)
}.launchIn(viewLifecycleOwner.lifecycleScope)
b.btnSend.setOnClickListener{ sendMessage() }
// FIX: MenuProvider en lugar de setHasOptionsMenu (obsoleto en Fragment 1.5+)
requireActivity().addMenuProvider(object:MenuProvider{
override fun onCreateMenu(menu:Menu,inf:MenuInflater){inf.inflate(R.menu.menu_chat,menu)}
override fun onMenuItemSelected(item:MenuItem): Boolean {
return when(item.itemId){
R.id.action_mute -> { toggleMute(); true }
R.id.action_schedule -> { showScheduleDialog(); true }
else -> false
}
}
},viewLifecycleOwner)
}
private fun sendMessage() {
val text=b.etMessage.text?.toString()?.trim()?:""
if(text.isBlank()) return
if(myNumber.isBlank()){Toast.makeText(requireContext(),"Cargando...",Toast.LENGTH_SHORT).show();
return}
lifecycleScope.launch {
try {
chatRepo.sendMessage(Message(from=myUid,fromNumber=myNumber,to=otherUid,toNumber=otherNumber,text=text
,delivered=true))
b.etMessage.setText("")
} catch(e:Exception){Toast.makeText(requireContext(),"Error: ${e.message}",Toast.LENGTH_SHORT).show()}
}
}
private fun isMuted():Boolean=requireContext()
.getSharedPreferences("muted_contacts",Context.MODE_PRIVATE)
```

```
.getBoolean("muted_$otherUid",false)
private fun toggleMute() {
val prefs=requireContext().getSharedPreferences("muted_contacts",Context.MODE_PRIVATE)
val cur=prefs.getBoolean("muted_$otherUid",false)
prefs.edit().putBoolean("muted_$otherUid",!cur).apply()
Toast.makeText(requireContext(),if(!cur)"Contacto silenciado" else "Notificaciones
activadas",Toast.LENGTH_SHORT).show()
}
private fun showScheduleDialog() {
val text=b.etMessage.text?.toString()?.trim()?:""
if(text.isBlank()){Toast.makeText(requireContext(),"Escribe el mensaje antes de
programarlo",Toast.LENGTH_SHORT).show(); return}
val cal=java.util.Calendar.getInstance()
android.app.DatePickerDialog(requireContext(),{_,y,m,d->
cal.set(y,m,d)
android.app.TimePickerDialog(requireContext(),{_,h,min->
cal.set(java.util.Calendar.HOUR_OF_DAY,h); cal.set(java.util.Calendar.MINUTE,min);
cal.set(java.util.Calendar.SECOND,0)
val delay=cal.timeInMillis-System.currentTimeMillis()
if(delay<=0){Toast.makeText(requireContext(),"Elige una hora futura",Toast.LENGTH_SHORT).show();
return@TimePickerDialog}
if(myNumber.isBlank()){Toast.makeText(requireContext(),"Espera que cargue tu
número",Toast.LENGTH_SHORT).show(); return@TimePickerDialog}
val data=androidx.work.Data.Builder()
.putString("from",myUid).putString("fromNumber",myNumber)
.putString("to",otherUid).putString("toNumber",otherNumber).putString("text",text).build()
androidx.work.WorkManager.getInstance(requireContext())
.enqueue(androidx.work.OneTimeWorkRequestBuilder<com.simplecallapp.workers.SendMessageWorker>()
.setInitialDelay(delay,java.util.concurrent.TimeUnit.MILLISECONDS)
.setInputData(data).build())
b.etMessage.setText("")
Toast.makeText(requireContext(),"Programado para ${java.text.SimpleDateFormat("dd/MM
HH:mm",java.util.Locale.getDefault()).format(cal.time)}",Toast.LENGTH_LONG).show()
},cal.get(java.util.Calendar.HOUR_OF_DAY),cal.get(java.util.Calendar.MINUTE),true).show()
},cal.get(java.util.Calendar.YEAR),cal.get(java.util.Calendar.MONTH),cal.get(java.util.Calendar.DAY_OF
_MONTH)).show()
}
override fun onDestroyView() { super.onDestroyView(); _b=null }
}
```

22. ContactsFragment.kt — Permiso RECORD_AUDIO Corregido

ui/contacts/ContactsFragment.kt

```
package com.simplecallapp.ui.contacts
import android.Manifest; import android.app.AlertDialog; import android.content.Intent
import android.content.pm.PackageManager; import android.os.Bundle; import android.view.*
import android.widget.*; import androidx.activity.result.contract.ActivityResultContracts
import androidx.core.content.ContextCompat; import androidx.core.os.bundleOf
import androidx.fragment.app.Fragment; import androidx.lifecycle.lifecycleScope
import androidx.navigation.fragment.findNavController
import androidx.recyclerview.widget.LinearLayoutManager
import com.google.firebase.auth.FirebaseAuth
import com.simplecallapp.R; import com.simplecallapp.data.model.Contact
import com.simplecallapp.data.repository.ContactRepository; import
com.simplecallapp.data.repository.UserRepository
import com.simplecallapp.databinding.FragmentContactsBinding; import
com.simplecallapp.ui.call.CallActivity
import kotlinx.coroutines.flow.launchIn; import kotlinx.coroutines.flow.onEach
import kotlinx.coroutines.launch; import java.util.UUID
class ContactsFragment : Fragment() {
private var _b: FragmentContactsBinding? = null
private val b get() = _b!!
private val contRepo=ContactRepository(); private val userRepo=UserRepository()
private lateinit var adapter: ContactsAdapter
private val myUid get()=FirebaseAuth.getInstance().currentUser?.uid?:""
private var pendingContact: Contact?=null
// FIX: pedir permiso RECORD_AUDIO antes de llamar
private val audioLauncher=registerForActivityResult(
ActivityResultContracts.RequestPermission()){ granted->
if(granted) pendingContact?.let{doCallContact(it)}
else Toast.makeText(requireContext(),"Permiso de micrófono requerido",Toast.LENGTH_LONG).show()
pendingContact=null
}
override fun onCreateView(i:LayoutInflater,c:ViewGroup?,s:Bundle?): View {
_b=FragmentContactsBinding.inflate(i,c,false); return b.root
}
override fun onViewCreated(v:View,s:Bundle?) {
super.onViewCreated(v,s)
adapter=ContactsAdapter(onCall={callContact(it)},onMessage={msgContact(it)})
b.recycler.layoutManager=LinearLayoutManager(requireContext())
b.recycler.adapter=adapter
contRepo.listenContacts(myUid).onEach{list->
adapter.submitList(list)
b.tvEmpty.visibility=if(list.isEmpty()) View.VISIBLE else View.GONE
}.launchIn(viewLifecycleOwner.lifecycleScope)
b.fabAddContact.setOnClickListener{showAddDialog()}
}
private fun callContact(c: Contact) {
val hasAudio=ContextCompat.checkSelfPermission(requireContext(),Manifest.permission.RECORD_AUDIO)==Pac
kageManager.PERMISSION_GRANTED
if(hasAudio) doCallContact(c) else { pendingContact=c;
audioLauncher.launch(Manifest.permission.RECORD_AUDIO) }
```

```
}
private fun doCallContact(c: Contact) {
lifecycleScope.launch {
try {
val r=userRepo.getUserByNumber(c.number)
if(r==null){Toast.makeText(requireContext(),"El número ${c.number} ya no está
registrado",Toast.LENGTH_SHORT).show(); return@launch}
startActivity(Intent(requireContext(),CallActivity::class.java).apply{
putExtra("CALL_ID",UUID.randomUUID().toString())
putExtra("IS_CALLER",true); putExtra("REMOTE_UID",r.uid); putExtra("REMOTE_NAME",c.name)})
} catch(e:Exception){Toast.makeText(requireContext(),"Error: ${e.message}",Toast.LENGTH_SHORT).show()}
}
}
private fun msgContact(c: Contact) {
lifecycleScope.launch {
try {
val r=userRepo.getUserByNumber(c.number)?:return@launch
findNavController().navigate(R.id.action_contacts_to_chat,
bundleOf("otherUid" to r.uid,"otherNumber" to c.number))
} catch(e:Exception){Toast.makeText(requireContext(),"Error: ${e.message}",Toast.LENGTH_SHORT).show()}
}
}
private fun showAddDialog() {
val ll=LinearLayout(requireContext()).apply{orientation=LinearLayout.VERTICAL;
setPadding(50,20,50,10)}
val etName=EditText(requireContext()).apply{hint="Nombre del contacto"}
val etNum=EditText(requireContext()).apply{hint="Número en la app";
inputType=android.text.InputType.TYPE_CLASS_NUMBER}
ll.addView(etName); ll.addView(etNum)
AlertDialog.Builder(requireContext()).setTitle("Agregar contacto").setView(ll)
.setPositiveButton("Guardar"){_,_->
val name=etName.text.toString().trim(); val num=etNum.text.toString().trim()
if(name.isBlank()||num.isBlank()) return@setPositiveButton
lifecycleScope.launch {
val r=userRepo.getUserByNumber(num)
if(r==null){Toast.makeText(requireContext(),"Número no registrado",Toast.LENGTH_SHORT).show();
return@launch}
contRepo.addContact(Contact(ownerUid=myUid,contactUid=r.uid,name=name,number=num))
}
}.setNegativeButton("Cancelar",null).show()
}
override fun onDestroyView() { super.onDestroyView(); _b=null }
}
```

23. ProfileFragment.kt — Logout + Cambiar Contraseña

ui/profile/ProfileFragment.kt

```
package com.simplecallapp.ui.profile
import android.app.AlertDialog; import android.content.Intent; import android.os.Bundle
import android.view.*; import android.widget.*
import androidx.fragment.app.Fragment; import androidx.lifecycle.lifecycleScope
import androidx.navigation.fragment.findNavController
import com.google.firebase.auth.FirebaseAuth
import com.simplecallapp.R; import com.simplecallapp.data.repository.UserRepository
import com.simplecallapp.databinding.FragmentProfileBinding; import
com.simplecallapp.ui.auth.LoginActivity
import kotlinx.coroutines.launch
class ProfileFragment : Fragment() {
private var _b: FragmentProfileBinding? = null
private val b get() = _b!!
private val auth=FirebaseAuth.getInstance(); private val userRepo=UserRepository()
override fun onCreateView(i:LayoutInflater,c:ViewGroup?,s:Bundle?): View {
_b=FragmentProfileBinding.inflate(i,c,false); return b.root
}
override fun onViewCreated(v:View,s:Bundle?) {
super.onViewCreated(v,s)
loadUser()
b.btnLogout.setOnClickListener {
AlertDialog.Builder(requireContext()).setTitle("Cerrar sesión")
.setMessage("¿Seguro que quieres salir?")
.setPositiveButton("Salir"){_,_->
auth.signOut()
startActivity(Intent(requireContext(),LoginActivity::class.java).apply{
flags=Intent.FLAG_ACTIVITY_NEW_TASK or Intent.FLAG_ACTIVITY_CLEAR_TASK})
requireActivity().finish()
}.setNegativeButton("Cancelar",null).show()
}
b.btnChangePassword.setOnClickListener { showChangePassDialog() }
b.btnNotifSettings.setOnClickListener {
findNavController().navigate(R.id.action_profile_to_notif_settings)
}
}
private fun loadUser() {
val fb=auth.currentUser?:return
b.tvEmail.text=fb.email?:""
lifecycleScope.launch {
val u=userRepo.getUser(fb.uid)
b.tvNumber.text=u?.phoneNumber?:"Sin número"
b.tvName.text=u?.displayName?:fb.email?:""
b.tvAvatar.text=(u?.displayName?:fb.email?:"U").firstOrNull()?.uppercaseChar()?.toString()?:"U"
}
}
private fun showChangePassDialog() {
val ll=LinearLayout(requireContext()).apply{orientation=LinearLayout.VERTICAL;
setPadding(50,20,50,10)}
```

```
val etCur=EditText(requireContext()).apply{hint="Contraseña actual";
inputType=android.text.InputType.TYPE_CLASS_TEXT or
android.text.InputType.TYPE_TEXT_VARIATION_PASSWORD}
val etNew=EditText(requireContext()).apply{hint="Nueva contraseña (6+ chars)";
inputType=android.text.InputType.TYPE_CLASS_TEXT or
android.text.InputType.TYPE_TEXT_VARIATION_PASSWORD}
ll.addView(etCur); ll.addView(etNew)
AlertDialog.Builder(requireContext()).setTitle("Cambiar contraseña").setView(ll)
.setPositiveButton("Cambiar"){_,_->
val cur=etCur.text.toString(); val new=etNew.text.toString()
if(cur.isBlank()||new.length<6){Toast.makeText(requireContext(),"Nueva contraseña: mínimo 6
chars",Toast.LENGTH_SHORT).show(); return@setPositiveButton}
val user=auth.currentUser?:return@setPositiveButton
val email=user.email?:return@setPositiveButton
val cred=com.google.firebase.auth.EmailAuthProvider.getCredential(email,cur)
user.reauthenticate(cred)
.addOnSuccessListener{ user.updatePassword(new)
.addOnSuccessListener{Toast.makeText(requireContext(),"Contraseña
actualizada",Toast.LENGTH_SHORT).show()}
.addOnFailureListener{e->Toast.makeText(requireContext(),"Error:
${e.message}",Toast.LENGTH_LONG).show()} }
.addOnFailureListener{Toast.makeText(requireContext(),"Contraseña actual
incorrecta",Toast.LENGTH_LONG).show()}
}.setNegativeButton("Cancelar",null).show()
}
override fun onDestroyView() { super.onDestroyView(); _b=null }
}
```

24-27. Fragments de Configuración

ui/settings/NotifSettingsFragment.kt

```
package com.simplecallapp.ui.settings
import android.content.Context; import android.os.Bundle; import android.view.*
import androidx.fragment.app.Fragment
import com.simplecallapp.databinding.FragmentNotifSettingsBinding
class NotifSettingsFragment : Fragment() {
private var _b: FragmentNotifSettingsBinding? = null
private val b get() = _b!!
companion object {
const val PREFS="notif_prefs"
const val KEY_CALLS="notif_calls"; const val KEY_MESSAGES="notif_messages"
const val KEY_SOUND="notif_sound"; const val KEY_VIBRATE="notif_vibrate"
fun areCallsEnabled(ctx:Context)=ctx.getSharedPreferences(PREFS,Context.MODE_PRIVATE).getBoolean(KEY_CALLS,true)
fun areMessagesEnabled(ctx:Context)=ctx.getSharedPreferences(PREFS,Context.MODE_PRIVATE).getBoolean(KEY_MESSAGES,true)
fun isSoundEnabled(ctx:Context)=ctx.getSharedPreferences(PREFS,Context.MODE_PRIVATE).getBoolean(KEY_SOUND,true)
fun isVibrateEnabled(ctx:Context)=ctx.getSharedPreferences(PREFS,Context.MODE_PRIVATE).getBoolean(KEY_VIBRATE,true)
}
override fun onCreateView(i:LayoutInflater,c:ViewGroup?,s:Bundle?): View {
_b=FragmentNotifSettingsBinding.inflate(i,c,false); return b.root
}
override fun onViewCreated(v:View,s:Bundle?) {
super.onViewCreated(v,s)
val p=requireContext().getSharedPreferences(PREFS,Context.MODE_PRIVATE)
b.switchCalls.isChecked=p.getBoolean(KEY_CALLS,true)
b.switchMessages.isChecked=p.getBoolean(KEY_MESSAGES,true)
b.switchSound.isChecked=p.getBoolean(KEY_SOUND,true)
b.switchVibrate.isChecked=p.getBoolean(KEY_VIBRATE,true)
b.switchCalls.setOnCheckedChangeListener{_,c->p.edit().putBoolean(KEY_CALLS,c).apply()}
b.switchMessages.setOnCheckedChangeListener{_,c->p.edit().putBoolean(KEY_MESSAGES,c).apply()}
b.switchSound.setOnCheckedChangeListener{_,c->p.edit().putBoolean(KEY_SOUND,c).apply()}
b.switchVibrate.setOnCheckedChangeListener{_,c->p.edit().putBoolean(KEY_VIBRATE,c).apply()}
}
override fun onDestroyView() { super.onDestroyView(); _b=null }
}
```

ui/settings/MessageSettingsFragment.kt — con getRingtoneUri() API 33+

```
package com.simplecallapp.ui.settings
import android.content.Context; import android.media.RingtoneManager; import android.net.Uri
import android.os.Build; import android.os.Bundle; import android.view.*
import androidx.activity.result.contract.ActivityResultContracts
import androidx.fragment.app.Fragment
import com.simplecallapp.databinding.FragmentMessageSettingsBinding
class MessageSettingsFragment : Fragment() {
private var _b: FragmentMessageSettingsBinding? = null
private val b get() = _b!!
companion object {
const val PREFS="msg_prefs"
const val KEY_READ="send_read_receipts"; const val KEY_RING="msg_ringtone_uri"
```

```
fun sendReadReceipts(ctx:Context)=ctx.getSharedPreferences(PREFS,Context.MODE_PRIVATE).getBoolean(KEY_
READ,true)
fun getMsgRingtoneUri(ctx:Context): Uri {
val saved=ctx.getSharedPreferences(PREFS,Context.MODE_PRIVATE).getString(KEY_RING,null)
return if(saved!=null) Uri.parse(saved)
else RingtoneManager.getDefaultUri(RingtoneManager.TYPE_NOTIFICATION)
}
}
// FIX: helper para getParcelableExtra compatible con API 33+
@Suppress("DEPRECATION")
private fun android.content.Intent?.getRingtoneUri(): Uri? {
this?:return null
return if(Build.VERSION.SDK_INT>=Build.VERSION_CODES.TIRAMISU)
getParcelableExtra(RingtoneManager.EXTRA_RINGTONE_PICKED_URI,Uri::class.java)
else getParcelableExtra(RingtoneManager.EXTRA_RINGTONE_PICKED_URI)
}
private val ringLauncher=registerForActivityResult(
ActivityResultContracts.StartActivityForResult()){ result->
val uri=result.data.getRingtoneUri()
if(uri!=null){
requireContext().getSharedPreferences(PREFS,Context.MODE_PRIVATE)
.edit().putString(KEY_RING,uri.toString()).apply()
b.tvMsgRingtone.text=RingtoneManager.getRingtone(requireContext(),uri).getTitle(requireContext())
}
}
override fun onCreateView(i:LayoutInflater,c:ViewGroup?,s:Bundle?): View {
_b=FragmentMessageSettingsBinding.inflate(i,c,false); return b.root
}
override fun onViewCreated(v:View,s:Bundle?) {
super.onViewCreated(v,s)
val p=requireContext().getSharedPreferences(PREFS,Context.MODE_PRIVATE)
b.switchReadReceipts.isChecked=p.getBoolean(KEY_READ,true)
b.switchReadReceipts.setOnCheckedChangeListener{_,c->p.edit().putBoolean(KEY_READ,c).apply()}
val uri=getMsgRingtoneUri(requireContext())
b.tvMsgRingtone.text=RingtoneManager.getRingtone(requireContext(),uri).getTitle(requireContext())
b.btnChooseMsgRingtone.setOnClickListener{
ringLauncher.launch(android.content.Intent(RingtoneManager.ACTION_RINGTONE_PICKER).apply{
putExtra(RingtoneManager.EXTRA_RINGTONE_TYPE,RingtoneManager.TYPE_NOTIFICATION)
putExtra(RingtoneManager.EXTRA_RINGTONE_SHOW_DEFAULT,true)
putExtra(RingtoneManager.EXTRA_RINGTONE_EXISTING_URI,uri)})
}
}
override fun onDestroyView() { super.onDestroyView(); _b=null }
}
```

ui/settings/CallSettingsFragment.kt — con getRingtoneUri() API 33+

```
package com.simplecallapp.ui.settings
import android.content.Context; import android.media.*; import android.net.Uri
import android.os.Build; import android.os.Bundle; import android.view.*; import
android.widget.SeekBar
import androidx.activity.result.contract.ActivityResultContracts
import androidx.fragment.app.Fragment
import com.simplecallapp.databinding.FragmentCallSettingsBinding
class CallSettingsFragment : Fragment() {
```

```
private var _b: FragmentCallSettingsBinding? = null
private val b get() = _b!!
companion object {
const val PREFS="call_prefs"; const val KEY_RING="call_ringtone_uri"
fun getCallRingtoneUri(ctx:Context): Uri {
val saved=ctx.getSharedPreferences(PREFS,Context.MODE_PRIVATE).getString(KEY_RING,null)
return if(saved!=null) Uri.parse(saved)
else RingtoneManager.getDefaultUri(RingtoneManager.TYPE_RINGTONE)
}
}
@Suppress("DEPRECATION")
private fun android.content.Intent?.getRingtoneUri(): Uri? {
this?:return null
return if(Build.VERSION.SDK_INT>=Build.VERSION_CODES.TIRAMISU)
getParcelableExtra(RingtoneManager.EXTRA_RINGTONE_PICKED_URI,Uri::class.java)
else getParcelableExtra(RingtoneManager.EXTRA_RINGTONE_PICKED_URI)
}
private val ringLauncher=registerForActivityResult(
ActivityResultContracts.StartActivityForResult()){ result->
val uri=result.data.getRingtoneUri()
if(uri!=null){
requireContext().getSharedPreferences(PREFS,Context.MODE_PRIVATE)
.edit().putString(KEY_RING,uri.toString()).apply()
b.tvCallRingtone.text=RingtoneManager.getRingtone(requireContext(),uri).getTitle(requireContext())
}
}
override fun onCreateView(i:LayoutInflater,c:ViewGroup?,s:Bundle?): View {
_b=FragmentCallSettingsBinding.inflate(i,c,false); return b.root
}
override fun onViewCreated(v:View,s:Bundle?) {
super.onViewCreated(v,s)
val am=requireContext().getSystemService(Context.AUDIO_SERVICE) as AudioManager
val uri=getCallRingtoneUri(requireContext())
b.tvCallRingtone.text=RingtoneManager.getRingtone(requireContext(),uri).getTitle(requireContext())
b.btnChooseCallRingtone.setOnClickListener{
ringLauncher.launch(android.content.Intent(RingtoneManager.ACTION_RINGTONE_PICKER).apply{
putExtra(RingtoneManager.EXTRA_RINGTONE_TYPE,RingtoneManager.TYPE_RINGTONE)
putExtra(RingtoneManager.EXTRA_RINGTONE_SHOW_DEFAULT,true)
putExtra(RingtoneManager.EXTRA_RINGTONE_EXISTING_URI,uri)})
}
val maxVol=am.getStreamMaxVolume(AudioManager.STREAM_RING)
b.seekVolume.max=maxVol; b.seekVolume.progress=am.getStreamVolume(AudioManager.STREAM_RING)
b.tvVolumeValue.text="${am.getStreamVolume(AudioManager.STREAM_RING)} / $maxVol"
b.seekVolume.setOnSeekBarChangeListener(object:SeekBar.OnSeekBarChangeListener{
override fun onProgressChanged(sb:SeekBar?,p:Int,fromUser:Boolean){
am.setStreamVolume(AudioManager.STREAM_RING,p,AudioManager.FLAG_PLAY_SOUND)
b.tvVolumeValue.text="$p / $maxVol"}
override fun onStartTrackingTouch(sb:SeekBar?){}
override fun onStopTrackingTouch(sb:SeekBar?){}})
}
override fun onDestroyView() { super.onDestroyView(); _b=null }
}
```

workers/SendMessageWorker.kt — con validación fromNumber

```
package com.simplecallapp.workers
import android.content.Context; import android.util.Log
import androidx.work.CoroutineWorker; import androidx.work.WorkerParameters
import com.simplecallapp.data.model.Message; import com.simplecallapp.data.repository.ChatRepository
class SendMessageWorker(ctx:Context, params:WorkerParameters):CoroutineWorker(ctx,params) {
override suspend fun doWork(): Result {
return try {
val from=inputData.getString("from")?:return Result.failure()
val fromNumber=inputData.getString("fromNumber")?:return Result.failure()
val to=inputData.getString("to")?:return Result.failure()
val toNumber=inputData.getString("toNumber")?:return Result.failure()
val text=inputData.getString("text")?:return Result.failure()
// FIX: no enviar si fromNumber está vacío
if(fromNumber.isBlank()){Log.e("SendMsgWorker","fromNumber vacío — descartado"); return
Result.failure()}
ChatRepository().sendMessage(Message(from=from,fromNumber=fromNumber,to=to,toNumber=toNumber,text=text
,delivered=true))
Result.success()
} catch(e:Exception){
if(runAttemptCount<3) Result.retry() else Result.failure()
}
}
}
```

28. MainActivity + Navigation Graph + Menus

ui/main/MainActivity.kt

```
package com.simplecallapp.ui.main
import android.os.Bundle
import androidx.appcompat.app.AppCompatActivity
import androidx.navigation.fragment.NavHostFragment
import androidx.navigation.ui.setupWithNavController
import com.simplecallapp.R; import com.simplecallapp.databinding.ActivityMainBinding
class MainActivity : AppCompatActivity() {
private lateinit var b: ActivityMainBinding
override fun onCreate(s:Bundle?) {
super.onCreate(s)
b=ActivityMainBinding.inflate(layoutInflater); setContentView(b.root)
val nav=(supportFragmentManager.findFragmentById(R.id.nav_host_fragment) as
NavHostFragment).navController
b.bottomNav.setupWithNavController(nav)
}
}
```

res/navigation/nav_graph.xml

```
<?xml version="1.0" encoding="utf-8"?>
<navigation xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:app="http://schemas.android.com/apk/res-auto"
android:id="@+id/nav_graph"
app:startDestination="@id/callsFragment">
<fragment android:id="@+id/callsFragment"
android:name="com.simplecallapp.ui.calls.CallsFragment" android:label="Llamadas">
<action android:id="@+id/action_calls_to_call_settings"
app:destination="@id/callSettingsFragment"/>
</fragment>
<fragment android:id="@+id/callSettingsFragment"
android:name="com.simplecallapp.ui.settings.CallSettingsFragment" android:label="Config. Llamadas"/>
<fragment android:id="@+id/conversationsFragment"
android:name="com.simplecallapp.ui.chat.ConversationsFragment" android:label="Mensajes">
<action android:id="@+id/action_conversations_to_chat" app:destination="@id/chatFragment"/>
<action android:id="@+id/action_conversations_to_msg_settings"
app:destination="@id/messageSettingsFragment"/>
</fragment>
<fragment android:id="@+id/chatFragment"
android:name="com.simplecallapp.ui.chat.ChatFragment" android:label="Chat">
<argument android:name="otherUid" app:argType="string"/>
<argument android:name="otherNumber" app:argType="string"/>
</fragment>
<fragment android:id="@+id/messageSettingsFragment"
android:name="com.simplecallapp.ui.settings.MessageSettingsFragment" android:label="Config.
Mensajes"/>
<fragment android:id="@+id/contactsFragment"
android:name="com.simplecallapp.ui.contacts.ContactsFragment" android:label="Contactos">
<action android:id="@+id/action_contacts_to_chat" app:destination="@id/chatFragment"/>
</fragment>
<fragment android:id="@+id/profileFragment"
android:name="com.simplecallapp.ui.profile.ProfileFragment" android:label="Perfil">
```

```
<action android:id="@+id/action_profile_to_notif_settings"
app:destination="@id/notifSettingsFragment"/>
</fragment>
<fragment android:id="@+id/notifSettingsFragment"
android:name="com.simplecallapp.ui.settings.NotifSettingsFragment" android:label="Notificaciones"/>
</navigation>
```

res/menu/bottom_nav_menu.xml

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android">
<item android:id="@+id/callsFragment" android:icon="@android:drawable/ic_menu_call"
android:title="Llamadas"/>
<item android:id="@+id/conversationsFragment" android:icon="@android:drawable/ic_dialog_email"
android:title="Mensajes"/>
<item android:id="@+id/contactsFragment" android:icon="@android:drawable/ic_menu_myplaces"
android:title="Contactos"/>
<item android:id="@+id/profileFragment" android:icon="@android:drawable/ic_menu_manage"
android:title="Perfil"/>
</menu>
```

res/menu/menu_chat.xml

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:app="http://schemas.android.com/apk/res-auto">
<item android:id="@+id/action_mute" android:title="Silenciar contacto" app:showAsAction="never"/>
<item android:id="@+id/action_schedule" android:title="Programar mensaje" app:showAsAction="never"/>
</menu>
```

29. SimpleCallApplication + Resources

SimpleCallApplication.kt

```
package com.simplecallapp
import android.app.Application; import com.google.firebase.FirebaseApp
class SimpleCallApplication : Application() {
override fun onCreate() { super.onCreate(); FirebaseApp.initializeApp(this) }
}
```

res/values/strings.xml

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
<string name="app_name">SimpleCallApp</string>
<!-- Web Client ID de Firebase Console → Auth → Google → Configuración SDK web -->
<string name="default_web_client_id">717516447001-koosm55a97p2h1hu7h87oe3id2rvhf1r.apps.googleusercontent.com</string>
</resources>
```

✓ El Web Client ID ya está configurado con el de tu proyecto: 717516447001-koosm55a97p2h1hu7h87oe3id2rvhf1r.apps.googleusercontent.com

30. Todos los Layouts XML

activity_login.xml — con sección de verificación de email

res/layout/activity_login.xml

```
<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:app="http://schemas.android.com/apk/res-auto"
android:layout_width="match_parent" android:layout_height="match_parent"
android:background="@color/background" android:fillViewport="true">
<LinearLayout android:layout_width="match_parent" android:layout_height="wrap_content"
android:orientation="vertical" android:gravity="center_horizontal"
android:paddingHorizontal="24dp" android:paddingTop="48dp" android:paddingBottom="32dp">
<ImageView android:src="@mipmap/ic_launcher"
android:layout_width="80dp" android:layout_height="80dp" android:layout_marginBottom="12dp"/>
<TextView android:text="SimpleCallApp" android:textSize="26sp" android:textStyle="bold"
android:textColor="@color/purple_500" android:layout_width="wrap_content"
android:layout_height="wrap_content" android:layout_marginBottom="4dp"/>
<TextView android:text="Llamadas y mensajes sobre WiFi" android:textSize="13sp"
android:textColor="@color/text_secondary" android:layout_width="wrap_content"
android:layout_height="wrap_content" android:layout_marginBottom="28dp"/>
<!-- SECCIÓN 1: Formulario de login -->
<LinearLayout android:id="@+id/layoutLoginForm" android:layout_width="match_parent"
android:layout_height="wrap_content" android:orientation="vertical">
<com.google.android.material.card.MaterialCardView android:layout_width="match_parent"
android:layout_height="wrap_content" app:cardCornerRadius="16dp"
app:cardElevation="4dp" android:layout_marginBottom="12dp">
<LinearLayout android:layout_width="match_parent" android:layout_height="wrap_content"
android:orientation="vertical" android:padding="20dp">
<com.google.android.material.textfield.TextInputLayout android:layout_width="match_parent"
android:layout_height="wrap_content" android:hint="Correo electrónico"
style="@style/Widget.Material3.TextInputLayout.OutlinedBox" android:layout_marginBottom="12dp">
<com.google.android.material.textfield.TextInputEditText android:id="@+id/etEmail"
android:layout_width="match_parent" android:layout_height="wrap_content"
android:inputType="textEmailAddress" android:imeOptions="actionNext"/>
</com.google.android.material.textfield.TextInputLayout>
<com.google.android.material.textfield.TextInputLayout android:layout_width="match_parent"
android:layout_height="wrap_content" android:hint="Contraseña"
app:endIconMode="password_toggle"
style="@style/Widget.Material3.TextInputLayout.OutlinedBox" android:layout_marginBottom="16dp">
<com.google.android.material.textfield.TextInputEditText android:id="@+id/etPassword"
android:layout_width="match_parent" android:layout_height="wrap_content"
android:inputType="textPassword" android:imeOptions="actionDone"/>
</com.google.android.material.textfield.TextInputLayout>
<Button android:id="@+id/btnLoginEmail" android:text="Iniciar sesión"
android:layout_width="match_parent" android:layout_height="52dp"
android:layout_marginBottom="8dp" app:cornerRadius="10dp"/>
<Button android:id="@+id/btnRegisterEmail" android:text="Crear cuenta nueva"
android:layout_width="match_parent" android:layout_height="52dp"
style="@style/Widget.Material3.Button.OutlinedButton"
app:cornerRadius="10dp" android:layout_marginBottom="4dp"/>
<Button android:id="@+id/btnForgotPassword" android:text="Olvidé mi contraseña"
android:layout_width="wrap_content" android:layout_height="wrap_content"
```

```
android:layout_gravity="center" style="@style/Widget.Material3.Button.TextButton"/>
</LinearLayout>
</com.google.android.material.card.MaterialCardView>
<LinearLayout android:layout_width="match_parent" android:layout_height="wrap_content"
android:orientation="horizontal" android:gravity="center_vertical"
android:layout_marginVertical="8dp">
<View android:layout_width="0dp" android:layout_height="1dp"
android:layout_weight="1" android:background="@color/purple_100"/>
<TextView android:text=" o continuar con " android:textColor="@color/text_secondary"
android:textSize="12sp" android:layout_width="wrap_content" android:layout_height="wrap_content"/>
<View android:layout_width="0dp" android:layout_height="1dp"
android:layout_weight="1" android:background="@color/purple_100"/>
</LinearLayout>
<com.google.android.material.card.MaterialCardView android:layout_width="match_parent"
android:layout_height="wrap_content" app:cardCornerRadius="10dp"
app:strokeWidth="1dp" app:strokeColor="@color/purple_100">
<Button android:id="@+id/btnGoogle" android:text="Continuar con Google"
android:layout_width="match_parent" android:layout_height="52dp"
style="@style/Widget.Material3.Button.TextButton"/>
</com.google.android.material.card.MaterialCardView>
</LinearLayout>
<!-- SECCIÓN 2: Verificación de email -->
<LinearLayout android:id="@+id/layoutVerification" android:layout_width="match_parent"
android:layout_height="wrap_content" android:orientation="vertical"
android:gravity="center" android:visibility="gone">
<ImageView android:src="@android:drawable/ic_dialog_email"
android:layout_width="64dp" android:layout_height="64dp" android:layout_marginBottom="16dp"/>
<TextView android:id="@+id/tvVerifTitle" android:textSize="22sp" android:textStyle="bold"
android:textColor="@color/purple_500" android:layout_width="wrap_content"
android:layout_height="wrap_content" android:layout_marginBottom="12dp"/>
<TextView android:id="@+id/tvVerifMsg" android:textSize="14sp"
android:textColor="@color/text_secondary" android:gravity="center"
android:layout_width="match_parent" android:layout_height="wrap_content"
android:layout_marginBottom="24dp"/>
<Button android:id="@+id/btnAlreadyVerified" android:text="Ya verifiqué — Entrar"
android:layout_width="match_parent" android:layout_height="52dp"
android:layout_marginBottom="10dp" app:cornerRadius="10dp"/>
<Button android:id="@+id/btnResendVerif" android:text="Reenviar email de verificación"
android:layout_width="match_parent" android:layout_height="wrap_content"
style="@style/Widget.Material3.Button.OutlinedButton" android:layout_marginBottom="8dp"/>
<Button android:id="@+id/btnBackToLogin" android:text="Volver al login"
android:layout_width="wrap_content" android:layout_height="wrap_content"
style="@style/Widget.Material3.Button.TextButton"/>
</LinearLayout>
<ProgressBar android:id="@+id/progressBar" android:visibility="gone"
android:layout_width="wrap_content" android:layout_height="wrap_content"
android:layout_marginTop="16dp"/>
</LinearLayout>
</ScrollView>
```

activity_choose_number.xml

res/layout/activity_choose_number.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:app="http://schemas.android.com/apk/res-auto"
android:layout_width="match_parent" android:layout_height="match_parent"
android:orientation="vertical" android:gravity="center" android:padding="32dp"
android:background="@color/background">
<ImageView android:src="@mipmap/ic_launcher"
android:layout_width="72dp" android:layout_height="72dp" android:layout_marginBottom="16dp"/>
<TextView android:text="Elige tu número" android:textSize="24sp" android:textStyle="bold"
android:textColor="@color/purple_500" android:layout_width="wrap_content"
android:layout_height="wrap_content" android:layout_marginBottom="8dp"/>
<TextView android:text="Otros usuarios lo usarán para llamarte y escribirte."
android:textSize="14sp" android:textColor="@color/text_secondary" android:gravity="center"
android:layout_width="match_parent" android:layout_height="wrap_content"
android:layout_marginBottom="24dp"/>
<EditText android:id="@+id/etNumber" android:hint="Ej: 12345 (4-12 dígitos)"
android:inputType="number" android:textSize="22sp" android:gravity="center"
android:layout_width="match_parent" android:layout_height="wrap_content"
android:layout_marginBottom="24dp"/>
<Button android:id="@+id/btnSave" android:text="Confirmar número"
android:layout_width="match_parent" android:layout_height="52dp" app:cornerRadius="10dp"/>
</LinearLayout>
```

activity_permission.xml

res/layout/activity_permission.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:app="http://schemas.android.com/apk/res-auto"
android:layout_width="match_parent" android:layout_height="match_parent"
android:orientation="vertical" android:gravity="center"
android:padding="32dp" android:background="@color/background">
<ImageView android:src="@mipmap/ic_launcher"
android:layout_width="80dp" android:layout_height="80dp" android:layout_marginBottom="24dp"/>
<TextView android:text="Permisos necesarios" android:textSize="24sp" android:textStyle="bold"
android:textColor="@color/purple_500" android:layout_width="wrap_content"
android:layout_height="wrap_content" android:layout_marginBottom="20dp"/>
<!-- Card Micrófono -->
<com.google.android.material.card.MaterialCardView android:layout_width="match_parent"
android:layout_height="wrap_content" app:cardCornerRadius="12dp"
app:cardElevation="2dp" android:layout_marginBottom="12dp">
<LinearLayout android:layout_width="match_parent" android:layout_height="wrap_content"
android:orientation="horizontal" android:padding="16dp" android:gravity="center_vertical">
<ImageView android:src="@android:drawable/ic_btn_speak_now"
android:layout_width="32dp" android:layout_height="32dp" android:layout_marginEnd="16dp"/>
<LinearLayout android:layout_width="0dp" android:layout_height="wrap_content"
android:layout_weight="1" android:orientation="vertical">
<TextView android:text="Micrófono" android:textStyle="bold" android:textSize="15sp"
android:layout_width="wrap_content" android:layout_height="wrap_content"/>
<TextView android:text="Para realizar y recibir llamadas" android:textSize="12sp"
android:textColor="@color/text_secondary" android:layout_width="wrap_content"
android:layout_height="wrap_content"/>
</LinearLayout>
<TextView android:text="Requerido" android:textSize="11sp" android:textColor="@color/red_500"
```

```
android:layout_width="wrap_content" android:layout_height="wrap_content"/>
</LinearLayout>
</com.google.android.material.card.MaterialCardView>
<!-- Card Notificaciones -->
<com.google.android.material.card.MaterialCardView android:layout_width="match_parent"
android:layout_height="wrap_content" app:cardCornerRadius="12dp"
app:cardElevation="2dp" android:layout_marginBottom="28dp">
<LinearLayout android:layout_width="match_parent" android:layout_height="wrap_content"
android:orientation="horizontal" android:padding="16dp" android:gravity="center_vertical">
<ImageView android:src="@android:drawable/ic_popup_reminder"
android:layout_width="32dp" android:layout_height="32dp" android:layout_marginEnd="16dp"/>
<LinearLayout android:layout_width="0dp" android:layout_height="wrap_content"
android:layout_weight="1" android:orientation="vertical">
<TextView android:text="Notificaciones" android:textStyle="bold" android:textSize="15sp"
android:layout_width="wrap_content" android:layout_height="wrap_content"/>
<TextView android:text="Para llamadas y mensajes entrantes" android:textSize="12sp"
android:textColor="@color/text_secondary" android:layout_width="wrap_content"
android:layout_height="wrap_content"/>
</LinearLayout>
<TextView android:text="Recomendado" android:textSize="11sp" android:textColor="@color/green_500"
android:layout_width="wrap_content" android:layout_height="wrap_content"/>
</LinearLayout>
</com.google.android.material.card.MaterialCardView>
<TextView android:id="@+id/tvStatus" android:text="" android:textSize="13sp"
android:gravity="center" android:layout_width="match_parent"
android:layout_height="wrap_content" android:layout_marginBottom="16dp"/>
<Button android:id="@+id/btnGrant" android:text="Conceder permisos"
android:layout_width="match_parent" android:layout_height="52dp"
android:layout_marginBottom="10dp" app:cornerRadius="10dp"/>
<Button android:id="@+id/btnSkip" android:text="Continuar sin notificaciones"
android:layout_width="match_parent" android:layout_height="wrap_content"
style="@style/Widget.Material3.Button.TextButton"/>
</LinearLayout>
```

activity_main.xml

res/layout/activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:app="http://schemas.android.com/apk/res-auto"
android:layout_width="match_parent" android:layout_height="match_parent"
android:orientation="vertical">
<androidx.fragment.app.FragmentContainerView android:id="@+id/nav_host_fragment"
android:name="androidx.navigation.fragment.NavHostFragment"
android:layout_width="match_parent" android:layout_height="0dp"
android:layout_weight="1" app:defaultNavHost="true" app:navGraph="@navigation/nav_graph"/>
<com.google.android.material.bottomnavigation.BottomNavigationView
android:id="@+id/bottomNav" android:layout_width="match_parent"
android:layout_height="wrap_content" app:menu="@menu/bottom_nav_menu"/>
</LinearLayout>
```

activity_call.xml — todos los IDs correctos

res/layout/activity_call.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="match_parent" android:layout_height="match_parent"
android:orientation="vertical" android:gravity="center"
android:background="@color/call_bg" android:padding="32dp">
<TextView android:id="@+id/tvAvatar" android:textSize="40sp" android:textStyle="bold"
android:textColor="@android:color/white" android:gravity="center"
android:background="@color/purple_500" android:layout_width="100dp"
android:layout_height="100dp" android:layout_marginBottom="20dp"/>
<TextView android:id="@+id/tvRemoteName" android:textSize="30sp" android:textStyle="bold"
android:textColor="@android:color/white" android:gravity="center"
android:layout_width="wrap_content" android:layout_height="wrap_content"
android:layout_marginBottom="8dp"/>
<TextView android:id="@+id/tvCallState" android:text="Conectando..."
android:textSize="15sp" android:textColor="#AAAAAA" android:gravity="center"
android:layout_width="wrap_content" android:layout_height="wrap_content"
android:layout_marginBottom="6dp"/>
<TextView android:id="@+id/tvDuration" android:text="00:00" android:textSize="32sp"
android:textColor="@android:color/white" android:fontFamily="monospace"
android:layout_width="wrap_content" android:layout_height="wrap_content"
android:layout_marginBottom="48dp"/>
<LinearLayout android:layout_width="match_parent" android:layout_height="wrap_content"
android:orientation="horizontal" android:gravity="center">
<!-- Mute -->
<LinearLayout android:layout_width="wrap_content" android:layout_height="wrap_content"
android:orientation="vertical" android:gravity="center" android:layout_marginHorizontal="20dp">
<ImageButton android:id="@+id/btnMute" android:src="@android:drawable/ic_btn_speak_now"
android:background="@drawable/circle_btn_bg"
android:layout_width="64dp" android:layout_height="64dp" android:layout_marginBottom="6dp"/>
<TextView android:id="@+id/tvMute" android:text="Silenciar" android:textSize="12sp"
android:textColor="#AAAAAA" android:layout_width="wrap_content" android:layout_height="wrap_content"/>
</LinearLayout>
<!-- Colgar -->
<LinearLayout android:layout_width="wrap_content" android:layout_height="wrap_content"
android:orientation="vertical" android:gravity="center">
<ImageButton android:id="@+id/btnHangup" android:src="@android:drawable/ic_menu_call"
android:backgroundTint="@color/red_500" android:background="@drawable/circle_btn_bg"
android:layout_width="80dp" android:layout_height="80dp" android:layout_marginBottom="6dp"/>
<TextView android:text="Colgar" android:textSize="12sp" android:textColor="#AAAAAA"
android:layout_width="wrap_content" android:layout_height="wrap_content"/>
</LinearLayout>
<!-- Altavoz -->
<LinearLayout android:layout_width="wrap_content" android:layout_height="wrap_content"
android:orientation="vertical" android:gravity="center" android:layout_marginHorizontal="20dp">
<ImageButton android:id="@+id/btnSpeaker" android:src="@android:drawable/ic_lock_silent_mode_off"
android:background="@drawable/circle_btn_bg"
android:layout_width="64dp" android:layout_height="64dp" android:layout_marginBottom="6dp"/>
<TextView android:id="@+id/tvSpeaker" android:text="Altavoz" android:textSize="12sp"
android:textColor="#AAAAAA" android:layout_width="wrap_content" android:layout_height="wrap_content"/>
</LinearLayout>
</LinearLayout>
</LinearLayout>
```

fragment_calls.xml — con botón de configuración

res/layout/fragment_calls.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="match_parent" android:layout_height="match_parent"
android:orientation="vertical">
<LinearLayout android:layout_width="match_parent" android:layout_height="wrap_content"
android:orientation="horizontal" android:gravity="center_vertical"
android:paddingHorizontal="8dp" android:paddingTop="4dp">
<com.google.android.material.tabs.TabLayout android:id="@+id/tabLayout"
android:layout_width="0dp" android:layout_height="wrap_content" android:layout_weight="1"/>
<ImageButton android:id="@+id/btnCallSettings"
android:src="@android:drawable/ic_menu_manage"
android:background="?attr/selectableItemBackgroundBorderless"
android:layout_width="40dp" android:layout_height="40dp"
android:contentDescription="Configuración de llamadas"/>
</LinearLayout>
<androidx.viewpager2.widget.ViewPager2 android:id="@+id/viewPager"
android:layout_width="match_parent" android:layout_height="0dp" android:layout_weight="1"/>
</LinearLayout>
```

fragment_dial.xml — CORREGIDO: ScrollView + TonalButton visibles

res/layout/fragment_dial.xml

```
<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:app="http://schemas.android.com/apk/res-auto"
android:layout_width="match_parent" android:layout_height="match_parent"
android:background="@color/background" android:fillViewport="true">
<LinearLayout android:layout_width="match_parent" android:layout_height="wrap_content"
android:orientation="vertical" android:gravity="center_horizontal" android:padding="16dp">
<TextView android:id="@+id/tvDialNumber" android:textSize="38sp" android:textStyle="bold"
android:textColor="@color/on_surface" android:gravity="center" android:minHeight="68dp"
android:layout_width="match_parent" android:layout_height="wrap_content"
android:layout_marginBottom="12dp"
android:hint="Ingresa un número" android:textColorHint="@color/text_secondary"/>
<LinearLayout android:layout_width="match_parent" android:layout_height="wrap_content"
android:orientation="horizontal" android:layout_marginBottom="4dp">
<Button android:id="@+id/btn1" android:text="1" android:textSize="22sp"
android:textColor="@color/on_surface" android:layout_width="0dp" android:layout_height="64dp"
android:layout_weight="1" android:layout_margin="3dp"
style="@style/Widget.Material3.Button.TonalButton"/>
<Button android:id="@+id/btn2" android:text="2" android:textSize="22sp"
android:textColor="@color/on_surface" android:layout_width="0dp" android:layout_height="64dp"
android:layout_weight="1" android:layout_margin="3dp"
style="@style/Widget.Material3.Button.TonalButton"/>
<Button android:id="@+id/btn3" android:text="3" android:textSize="22sp"
android:textColor="@color/on_surface" android:layout_width="0dp" android:layout_height="64dp"
android:layout_weight="1" android:layout_margin="3dp"
style="@style/Widget.Material3.Button.TonalButton"/>
</LinearLayout>
<LinearLayout android:layout_width="match_parent" android:layout_height="wrap_content"
android:orientation="horizontal" android:layout_marginBottom="4dp">
```

```
<Button android:id="@+id/btn4" android:text="4" android:textSize="22sp"
android:textColor="@color/on_surface" android:layout_width="0dp" android:layout_height="64dp"
android:layout_weight="1" android:layout_margin="3dp"
style="@style/Widget.Material3.Button.TonalButton"/>

<Button android:id="@+id/btn5" android:text="5" android:textSize="22sp"
android:textColor="@color/on_surface" android:layout_width="0dp" android:layout_height="64dp"
android:layout_weight="1" android:layout_margin="3dp"
style="@style/Widget.Material3.Button.TonalButton"/>

<Button android:id="@+id/btn6" android:text="6" android:textSize="22sp"
android:textColor="@color/on_surface" android:layout_width="0dp" android:layout_height="64dp"
android:layout_weight="1" android:layout_margin="3dp"
style="@style/Widget.Material3.Button.TonalButton"/>

</LinearLayout>

<LinearLayout android:layout_width="match_parent" android:layout_height="wrap_content"
android:orientation="horizontal" android:layout_marginBottom="4dp">

<Button android:id="@+id/btn7" android:text="7" android:textSize="22sp"
android:textColor="@color/on_surface" android:layout_width="0dp" android:layout_height="64dp"
android:layout_weight="1" android:layout_margin="3dp"
style="@style/Widget.Material3.Button.TonalButton"/>

<Button android:id="@+id/btn8" android:text="8" android:textSize="22sp"
android:textColor="@color/on_surface" android:layout_width="0dp" android:layout_height="64dp"
android:layout_weight="1" android:layout_margin="3dp"
style="@style/Widget.Material3.Button.TonalButton"/>

<Button android:id="@+id/btn9" android:text="9" android:textSize="22sp"
android:textColor="@color/on_surface" android:layout_width="0dp" android:layout_height="64dp"
android:layout_weight="1" android:layout_margin="3dp"
style="@style/Widget.Material3.Button.TonalButton"/>

</LinearLayout>

<LinearLayout android:layout_width="match_parent" android:layout_height="wrap_content"
android:orientation="horizontal" android:layout_marginBottom="12dp">

<Button android:id="@+id/btnStar" android:text="*" android:textSize="28sp"
android:textColor="@color/on_surface" android:layout_width="0dp" android:layout_height="64dp"
android:layout_weight="1" android:layout_margin="3dp"
style="@style/Widget.Material3.Button.TonalButton"/>

<Button android:id="@+id/btn0" android:text="0" android:textSize="22sp"
android:textColor="@color/on_surface" android:layout_width="0dp" android:layout_height="64dp"
android:layout_weight="1" android:layout_margin="3dp"
style="@style/Widget.Material3.Button.TonalButton"/>

<Button android:id="@+id/btnBackspace" android:text="&#x232B;" android:textSize="22sp"
android:textColor="@color/on_surface" android:layout_width="0dp" android:layout_height="64dp"
android:layout_weight="1" android:layout_margin="3dp"
style="@style/Widget.Material3.Button.TonalButton"/>

</LinearLayout>

<Button android:id="@+id/btnCall" android:text="Llamar" android:textSize="18sp"

android:textColor="@android:color/white" android:backgroundTint="@color/green_500"

android:layout_width="200dp" android:layout_height="64dp" android:layout_gravity="center"

android:layout_marginTop="4dp" android:layout_marginBottom="16dp" app:cornerRadius="32dp"/>

</LinearLayout>

</ScrollView>
```

fragment_call_history.xml

res/layout/fragment_call_history.xml

```
<?xml version="1.0" encoding="utf-8"?>
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="match_parent" android:layout_height="match_parent">
<androidx.recyclerview.widget.RecyclerView android:id="@+id/recycler"
android:layout_width="match_parent" android:layout_height="match_parent" android:padding="8dp"/>
<TextView android:id="@+id/tvEmpty" android:text="Sin llamadas recientes"
android:visibility="gone" android:gravity="center" android:textColor="#888888"
android:textSize="16sp" android:layout_width="match_parent" android:layout_height="match_parent"/>
</FrameLayout>
```

fragment_conversations.xml

res/layout/fragment_conversations.xml

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.coordinatorlayout.widget.CoordinatorLayout
xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:app="http://schemas.android.com/apk/res-auto"
android:layout_width="match_parent" android:layout_height="match_parent">
<androidx.recyclerview.widget.RecyclerView android:id="@+id/recycler"
android:layout_width="match_parent" android:layout_height="match_parent" android:padding="8dp"/>
<TextView android:id="@+id/tvEmpty" android:text="Sin conversaciones. Toca + para empezar."
android:visibility="gone" android:gravity="center" android:textColor="#888888"
android:textSize="16sp" android:layout_width="match_parent" android:layout_height="match_parent"/>
<com.google.android.material.floatingactionbutton.FloatingActionButton
android:id="@+id/fabNewChat" android:src="@android:drawable/ic_input_add"
android:layout_width="wrap_content" android:layout_height="wrap_content"
android:layout_gravity="bottom|end" android:layout_margin="16dp"/>
</androidx.coordinatorlayout.widget.CoordinatorLayout>
```

fragment_chat.xml

res/layout/fragment_chat.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="match_parent" android:layout_height="match_parent"
android:orientation="vertical">
<TextView android:id="@+id/tvTitle" android:layout_width="match_parent"
android:layout_height="56dp" android:textSize="18sp" android:textStyle="bold"
android:gravity="center_vertical" android:paddingStart="16dp"
android:background="?attr/colorPrimary" android:textColor="@android:color/white"/>
<androidx.recyclerview.widget.RecyclerView android:id="@+id/recycler"
android:layout_width="match_parent" android:layout_height="0dp"
android:layout_weight="1" android:padding="8dp" android:clipToPadding="false"/>
<LinearLayout android:layout_width="match_parent" android:layout_height="wrap_content"
android:orientation="horizontal" android:padding="8dp">
<EditText android:id="@+id/etMessage" android:hint="Escribe un mensaje..."
android:maxLines="4" android:layout_width="0dp" android:layout_height="wrap_content"
android:layout_weight="1" android:layout_marginEnd="8dp"/>
<Button android:id="@+id/btnSend" android:text="Enviar"
android:layout_width="wrap_content" android:layout_height="wrap_content"/>
</LinearLayout>
```

```
</LinearLayout>
```

fragment_contacts.xml

res/layout/fragment_contacts.xml

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.coordinatorlayout.widget.CoordinatorLayout
xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:app="http://schemas.android.com/apk/res-auto"
android:layout_width="match_parent" android:layout_height="match_parent">
<androidx.recyclerview.widget.RecyclerView android:id="@+id/recycler"
android:layout_width="match_parent" android:layout_height="match_parent" android:padding="8dp"/>
<TextView android:id="@+id/tvEmpty" android:text="Sin contactos. Toca + para agregar."
android:visibility="gone" android:gravity="center" android:textColor="#888888"
android:textSize="16sp" android:layout_width="match_parent" android:layout_height="match_parent"/>
<com.google.android.material.floatingactionbutton.FloatingActionButton
android:id="@+id/fabAddContact" android:src="@android:drawable/ic_input_add"
android:layout_width="wrap_content" android:layout_height="wrap_content"
android:layout_gravity="bottom|end" android:layout_margin="16dp"/>
</androidx.coordinatorlayout.widget.CoordinatorLayout>
```

fragment_profile.xml

res/layout/fragment_profile.xml

```
<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:app="http://schemas.android.com/apk/res-auto"
android:layout_width="match_parent" android:layout_height="match_parent"
android:background="@color/background">
<LinearLayout android:layout_width="match_parent" android:layout_height="wrap_content"
android:orientation="vertical" android:padding="16dp">
<com.google.android.material.card.MaterialCardView android:layout_width="match_parent"
android:layout_height="wrap_content" app:cardCornerRadius="16dp"
app:cardElevation="4dp" android:layout_marginBottom="12dp">
<LinearLayout android:layout_width="match_parent" android:layout_height="wrap_content"
android:orientation="vertical" android:padding="20dp" android:gravity="center">
<TextView android:id="@+id/tvAvatar" android:text="U" android:textSize="28sp"
android:textStyle="bold" android:textColor="@android:color/white"
android:gravity="center" android:background="@color/purple_500"
android:layout_width="72dp" android:layout_height="72dp" android:layout_marginBottom="12dp"/>
<TextView android:id="@+id/tvName" android:textSize="20sp" android:textStyle="bold"
android:textColor="@color/on_surface" android:layout_width="wrap_content"
android:layout_height="wrap_content"/>
<TextView android:id="@+id/tvEmail" android:textSize="13sp"
android:textColor="@color/text_secondary" android:layout_width="wrap_content"
android:layout_height="wrap_content" android:layout_marginTop="4dp"/>
</LinearLayout>
</com.google.android.material.card.MaterialCardView>
<com.google.android.material.card.MaterialCardView android:layout_width="match_parent"
android:layout_height="wrap_content" app:cardCornerRadius="12dp"
app:cardElevation="2dp" android:layout_marginBottom="12dp">
<LinearLayout android:layout_width="match_parent" android:layout_height="wrap_content"
android:orientation="horizontal" android:padding="16dp" android:gravity="center_vertical">
<ImageView android:src="@android:drawable/ic_menu_call"
```

```
android:layout_width="24dp" android:layout_height="24dp" android:layout_marginEnd="12dp"/>
<LinearLayout android:layout_width="0dp" android:layout_height="wrap_content"
android:layout_weight="1" android:orientation="vertical">
<TextView android:text="Mi número en la app" android:textSize="12sp"
android:textColor="@color/text_secondary" android:layout_width="wrap_content"
android:layout_height="wrap_content"/>
<TextView android:id="@+id/tvNumber" android:textSize="22sp" android:textStyle="bold"
android:textColor="@color/purple_500" android:layout_width="wrap_content"
android:layout_height="wrap_content"/>
</LinearLayout>
</LinearLayout>
</com.google.android.material.card.MaterialCardView>
<Button android:id="@+id/btnNotifSettings" android:text="Configurar notificaciones"
android:layout_width="match_parent" android:layout_height="52dp"
style="@style/Widget.Material3.Button.OutlinedButton"
android:layout_marginBottom="10dp" app:cornerRadius="10dp"/>
<Button android:id="@+id/btnChangePassword" android:text="Cambiar contraseña"
android:layout_width="match_parent" android:layout_height="52dp"
style="@style/Widget.Material3.Button.OutlinedButton"
android:layout_marginBottom="10dp" app:cornerRadius="10dp"/>
<Button android:id="@+id/btnLogout" android:text="Cerrar sesión"
android:backgroundTint="@color/red_500" android:textColor="@android:color/white"
android:layout_width="match_parent" android:layout_height="52dp" app:cornerRadius="10dp"/>
</LinearLayout>
</ScrollView>
```

Items para RecyclerViews

res/layout/item_message_sent.xml — con tvTick

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="match_parent" android:layout_height="wrap_content"
android:orientation="vertical" android:gravity="end" android:padding="4dp">
<LinearLayout android:layout_width="wrap_content" android:layout_height="wrap_content"
android:orientation="vertical" android:background="#DCF8C6" android:padding="10dp"
android:maxWidth="260dp">
<TextView android:id="@+id/tvText" android:textSize="15sp" android:textColor="#000000"
android:layout_width="wrap_content" android:layout_height="wrap_content"/>
<LinearLayout android:layout_width="wrap_content" android:layout_height="wrap_content"
android:orientation="horizontal" android:layout_gravity="end" android:layout_marginTop="2dp">
<TextView android:id="@+id/tvTime" android:textSize="11sp" android:textColor="#888888"
android:layout_width="wrap_content" android:layout_height="wrap_content"
android:layout_marginEnd="4dp"/>
<TextView android:id="@+id/tvTick" android:text="✓" android:textSize="12sp"
android:textColor="#9E9E9E" android:layout_width="wrap_content" android:layout_height="wrap_content"/>
</LinearLayout>
</LinearLayout>
</LinearLayout>
```

res/layout/item_message_received.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="match_parent" android:layout_height="wrap_content"
android:orientation="vertical" android:gravity="start" android:padding="4dp">
<LinearLayout android:layout_width="wrap_content" android:layout_height="wrap_content"
```

```
android:orientation="vertical" android:background="#FFFFFF" android:padding="10dp"
android:maxWidth="260dp">
<TextView android:id="@+id/tvText" android:textSize="15sp" android:textColor="#000000"
android:layout_width="wrap_content" android:layout_height="wrap_content"/>
<TextView android:id="@+id/tvTime" android:textSize="11sp" android:textColor="#888888"
android:layout_width="wrap_content" android:layout_height="wrap_content"
android:layout_gravity="end" android:layout_marginTop="2dp"/>
</LinearLayout>
</LinearLayout>
```

res/layout/item_contact.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="match_parent" android:layout_height="wrap_content"
android:orientation="horizontal" android:padding="12dp" android:gravity="center_vertical">
<LinearLayout android:layout_width="0dp" android:layout_height="wrap_content"
android:layout_weight="1" android:orientation="vertical">
<TextView android:id="@+id/tvName" android:textSize="16sp" android:textStyle="bold"
android:layout_width="wrap_content" android:layout_height="wrap_content"/>
<TextView android:id="@+id/tvNumber" android:textSize="13sp"
android:textColor="@color/text_secondary" android:layout_width="wrap_content"
android:layout_height="wrap_content"/>
</LinearLayout>
<Button android:id="@+id/btnCall" android:text="Llamar"
android:backgroundTint="@color/green_500" android:textColor="@android:color/white"
android:layout_width="wrap_content" android:layout_height="wrap_content"
android:layout_marginEnd="8dp"/>
<Button android:id="@+id/btnMessage" android:text="Mensaje"
android:layout_width="wrap_content" android:layout_height="wrap_content"
style="@style/Widget.Material3.Button.OutlinedButton"/>
</LinearLayout>
```

res/layout/item_call_history.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="match_parent" android:layout_height="wrap_content"
android:orientation="horizontal" android:padding="12dp" android:gravity="center_vertical">
<LinearLayout android:layout_width="0dp" android:layout_height="wrap_content"
android:layout_weight="1" android:orientation="vertical">
<TextView android:id="@+id/tvName" android:textSize="16sp" android:textStyle="bold"
android:layout_width="wrap_content" android:layout_height="wrap_content"/>
<TextView android:id="@+id/tvType" android:textSize="13sp" android:textColor="@color/text_secondary"
android:layout_width="wrap_content" android:layout_height="wrap_content"/>
</LinearLayout>
<LinearLayout android:layout_width="wrap_content" android:layout_height="wrap_content"
android:orientation="vertical" android:gravity="end">
<TextView android:id="@+id/tvDate" android:textSize="12sp" android:textColor="@color/text_secondary"
android:layout_width="wrap_content" android:layout_height="wrap_content"/>
<TextView android:id="@+id/tvDuration" android:textSize="12sp"
android:textColor="@color/text_secondary"
android:layout_width="wrap_content" android:layout_height="wrap_content"/>
</LinearLayout>
</LinearLayout>
```

res/layout/item_conversation.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="match_parent" android:layout_height="wrap_content"
android:orientation="horizontal" android:padding="12dp" android:gravity="center_vertical">
<LinearLayout android:layout_width="0dp" android:layout_height="wrap_content"
android:layout_weight="1" android:orientation="vertical">
<TextView android:id="@+id/tvName" android:textSize="16sp" android:textStyle="bold"
android:layout_width="wrap_content" android:layout_height="wrap_content"/>
<TextView android:id="@+id/tvLastMessage" android:textSize="13sp"
android:textColor="@color/text_secondary" android:maxLines="1" android:ellipsize="end"
android:layout_width="wrap_content" android:layout_height="wrap_content"/>
</LinearLayout>
<TextView android:id="@+id/tvTime" android:textSize="12sp" android:textColor="@color/text_secondary"
android:layout_width="wrap_content" android:layout_height="wrap_content"/>
</LinearLayout>
```

res/drawable/circle_btn_bg.xml

```
<?xml version="1.0" encoding="utf-8"?>
<shape xmlns:android="http://schemas.android.com/apk/res/android" android:shape="oval">
<solid android:color="#2A2A4A"/>
</shape>
```

Layouts de configuración restantes (simplificados pero funcionales)

res/layout/fragment_notif_settings.xml

```
<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:app="http://schemas.android.com/apk/res-auto"
android:layout_width="match_parent" android:layout_height="match_parent"
android:background="@color/background">
<LinearLayout android:layout_width="match_parent" android:layout_height="wrap_content"
android:orientation="vertical" android:padding="16dp">
<TextView android:text="Notificaciones" android:textSize="22sp" android:textStyle="bold"
android:textColor="@color/purple_500" android:layout_width="wrap_content"
android:layout_height="wrap_content" android:layout_marginBottom="16dp"/>
<com.google.android.material.card.MaterialCardView android:layout_width="match_parent"
android:layout_height="wrap_content" app:cardCornerRadius="12dp" app:cardElevation="2dp"
android:layout_marginBottom="12dp">
<LinearLayout android:layout_width="match_parent" android:layout_height="wrap_content"
android:orientation="vertical" android:padding="8dp">
<LinearLayout android:layout_width="match_parent" android:layout_height="56dp"
android:orientation="horizontal" android:gravity="center_vertical" android:paddingHorizontal="8dp">
<TextView android:text="Llamadas entrantes" android:textSize="15sp"
android:layout_width="0dp" android:layout_height="wrap_content" android:layout_weight="1"/>
<com.google.android.material.switchmaterial.SwitchMaterial android:id="@+id/switchCalls"
android:layout_width="wrap_content" android:layout_height="wrap_content"/>
</LinearLayout>
<LinearLayout android:layout_width="match_parent" android:layout_height="56dp"
android:orientation="horizontal" android:gravity="center_vertical" android:paddingHorizontal="8dp">
<TextView android:text="Mensajes nuevos" android:textSize="15sp"
android:layout_width="0dp" android:layout_height="wrap_content" android:layout_weight="1"/>
<com.google.android.material.switchmaterial.SwitchMaterial android:id="@+id/switchMessages"
android:layout_width="wrap_content" android:layout_height="wrap_content"/>
</LinearLayout>
```

```
<LinearLayout android:layout_width="match_parent" android:layout_height="56dp"
android:orientation="horizontal" android:gravity="center_vertical" android:paddingHorizontal="8dp">
<TextView android:text="Sonido de llamada" android:textSize="15sp"
android:layout_width="0dp" android:layout_height="wrap_content" android:layout_weight="1"/>
<com.google.android.material.switchmaterial.SwitchMaterial android:id="@+id/switchSound"
android:layout_width="wrap_content" android:layout_height="wrap_content"/>
</LinearLayout>
<LinearLayout android:layout_width="match_parent" android:layout_height="56dp"
android:orientation="horizontal" android:gravity="center_vertical" android:paddingHorizontal="8dp">
<TextView android:text="Vibración" android:textSize="15sp"
android:layout_width="0dp" android:layout_height="wrap_content" android:layout_weight="1"/>
<com.google.android.material.switchmaterial.SwitchMaterial android:id="@+id/switchVibrate"
android:layout_width="wrap_content" android:layout_height="wrap_content"/>
</LinearLayout>
</LinearLayout>
</com.google.android.material.card.MaterialCardView>
</LinearLayout>
</ScrollView>
```

res/layout/fragment_message_settings.xml

```
<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:app="http://schemas.android.com/apk/res-auto"
android:layout_width="match_parent" android:layout_height="match_parent"
android:background="@color/background">
<LinearLayout android:layout_width="match_parent" android:layout_height="wrap_content"
android:orientation="vertical" android:padding="16dp">
<TextView android:text="Configuración de Mensajes" android:textSize="20sp" android:textStyle="bold"
android:textColor="@color/purple_500" android:layout_width="wrap_content"
android:layout_height="wrap_content" android:layout_marginBottom="16dp"/>
<com.google.android.material.card.MaterialCardView android:layout_width="match_parent"
android:layout_height="wrap_content" app:cardCornerRadius="12dp" app:cardElevation="2dp"
android:layout_marginBottom="12dp">
<LinearLayout android:layout_width="match_parent" android:layout_height="wrap_content"
android:orientation="horizontal" android:padding="16dp" android:gravity="center_vertical">
<LinearLayout android:layout_width="0dp" android:layout_height="wrap_content"
android:layout_weight="1" android:orientation="vertical">
<TextView android:text="Confirmaciones de lectura" android:textSize="15sp"
android:layout_width="wrap_content" android:layout_height="wrap_content"/>
<TextView android:text="Mostrar a otros cuando lees sus mensajes"
android:textSize="12sp" android:textColor="@color/text_secondary"
android:layout_width="wrap_content" android:layout_height="wrap_content"/>
</LinearLayout>
<com.google.android.material.switchmaterial.SwitchMaterial android:id="@+id/switchReadReceipts"
android:layout_width="wrap_content" android:layout_height="wrap_content"/>
</LinearLayout>
</com.google.android.material.card.MaterialCardView>
<com.google.android.material.card.MaterialCardView android:layout_width="match_parent"
android:layout_height="wrap_content" app:cardCornerRadius="12dp" app:cardElevation="2dp">
<LinearLayout android:layout_width="match_parent" android:layout_height="wrap_content"
android:orientation="vertical" android:padding="16dp">
<TextView android:text="Tono de notificación" android:textSize="15sp" android:textStyle="bold"
android:layout_width="wrap_content" android:layout_height="wrap_content"
android:layout_marginBottom="8dp"/>
```

```
<TextView android:id="@+id/tvMsgRingtone" android:textSize="13sp"
android:textColor="@color/text_secondary" android:layout_width="wrap_content"
android:layout_height="wrap_content" android:layout_marginBottom="10dp"/>
<Button android:id="@+id/btnChooseMsgRingtone" android:text="Elegir tono"
android:layout_width="wrap_content" android:layout_height="wrap_content"
style="@style/Widget.Material3.Button.OutlinedButton" app:cornerRadius="8dp"/>
</LinearLayout>
</com.google.android.material.card.MaterialCardView>
</LinearLayout>
</ScrollView>
```

res/layout/fragment_call_settings.xml

```
<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:app="http://schemas.android.com/apk/res-auto"
android:layout_width="match_parent" android:layout_height="match_parent"
android:background="@color/background">
<LinearLayout android:layout_width="match_parent" android:layout_height="wrap_content"
android:orientation="vertical" android:padding="16dp">
<TextView android:text="Configuración de Llamadas" android:textSize="20sp" android:textStyle="bold"
android:textColor="@color/purple_500" android:layout_width="wrap_content"
android:layout_height="wrap_content" android:layout_marginBottom="16dp"/>
<com.google.android.material.card.MaterialCardView android:layout_width="match_parent"
android:layout_height="wrap_content" app:cardCornerRadius="12dp" app:cardElevation="2dp"
android:layout_marginBottom="12dp">
<LinearLayout android:layout_width="match_parent" android:layout_height="wrap_content"
android:orientation="vertical" android:padding="16dp">
<TextView android:text="Tono de llamada entrante" android:textSize="15sp" android:textStyle="bold"
android:layout_width="wrap_content" android:layout_height="wrap_content"
android:layout_marginBottom="8dp"/>
<TextView android:id="@+id/tvCallRingtone" android:textSize="13sp"
android:textColor="@color/text_secondary" android:layout_width="wrap_content"
android:layout_height="wrap_content" android:layout_marginBottom="10dp"/>
<Button android:id="@+id/btnChooseCallRingtone" android:text="Elegir tono de llamada"
android:layout_width="wrap_content" android:layout_height="wrap_content"
style="@style/Widget.Material3.Button.OutlinedButton" app:cornerRadius="8dp"/>
</LinearLayout>
</com.google.android.material.card.MaterialCardView>
<com.google.android.material.card.MaterialCardView android:layout_width="match_parent"
android:layout_height="wrap_content" app:cardCornerRadius="12dp" app:cardElevation="2dp">
<LinearLayout android:layout_width="match_parent" android:layout_height="wrap_content"
android:orientation="vertical" android:padding="16dp">
<LinearLayout android:layout_width="match_parent" android:layout_height="wrap_content"
android:orientation="horizontal" android:gravity="center_vertical" android:layout_marginBottom="8dp">
<TextView android:text="Volumen del ringtone" android:textSize="15sp" android:textStyle="bold"
android:layout_width="0dp" android:layout_height="wrap_content" android:layout_weight="1"/>
<TextView android:id="@+id/tvVolumeValue" android:textSize="13sp"
android:textColor="@color/text_secondary" android:layout_width="wrap_content"
android:layout_height="wrap_content"/>
</LinearLayout>
<SeekBar android:id="@+id/seekVolume" android:layout_width="match_parent"
android:layout_height="wrap_content"/>
</LinearLayout>
</com.google.android.material.card.MaterialCardView>
```

```
</LinearLayout>
</ScrollView>
```

31. Reglas de Seguridad Firestore

i Ir a Firebase Console → Firestore → Reglas → Publicar. Reemplazar todo.

firestore.rules

```
rules_version = '2';
service cloud.firestore {
match /databases/{database}/documents {
match /users/{uid} {
allow read: if request.auth != null;
allow write: if request.auth != null && request.auth.uid == uid;
}
match /messages/{msgId} {
allow read: if request.auth != null &&
(resource.data.from == request.auth.uid || resource.data.to == request.auth.uid);
allow create: if request.auth != null &&
request.resource.data.from == request.auth.uid;
// Permitir actualizar solo el campo 'read' (confirmación de lectura)
allow update: if request.auth != null &&
resource.data.to == request.auth.uid &&
request.resource.data.diff(resource.data).affectedKeys().hasOnly(['read']);
allow delete: if false;
}
match /calls/{callId} {
allow read, write: if request.auth != null;
match /callerCandidates/{c} { allow read, write: if request.auth != null; }
match /receiverCandidates/{c} { allow read, write: if request.auth != null; }
}
match /call_history/{hid} {
allow read: if request.auth != null &&
(resource.data.callerUid == request.auth.uid ||
resource.data.receiverUid == request.auth.uid);
allow create: if request.auth != null;
allow update, delete: if false;
}
match /contacts/{cid} {
allow read, write: if request.auth != null &&
resource.data.ownerUid == request.auth.uid;
allow create: if request.auth != null &&
request.resource.data.ownerUid == request.auth.uid;
}
}
}
```

32. Generar el APK

Lista de verificación antes de compilar

1. google-services.json (sección 2) está en app/google-services.json
2. strings.xml tiene el Web Client ID:
717516447001-koosm55a97p2h1hu7h87oe3id2rvhf1r.apps.googleusercontent.com
3. WebRTCClient.kt: reemplazar 'openrelayproject' con credenciales de app.metered.ca
4. Todos los archivos Kotlin en sus paquetes correctos (com.simplecallapp.xxx)
5. Todos los XML en res/layout/, res/navigation/, res/menu/
6. Los 2 archivos adaptive-icon en res/mipmap-anydpi-v26/
7. Reglas Firestore publicadas e índices compuestos creados (sección 4)

Compilar y probar

1. File → Sync Project with Gradle Files → esperar que termine
2. Build → Clean Project
3. Build → Build Bundle(s)/APK(s) → Build APK(s)
4. APK en: app/build/outputs/apk/debug/app-debug.apk
5. Instalar en 2 dispositivos físicos para probar llamadas

33. Checklist Final de Pruebas

✔ Auth

- ■ Login con Google funciona
- ■ Registro con email envía email de verificación
- ■ Sin verificar email: no puede entrar (muestra pantalla de verificación)
- ■ Reenviar verificación funciona
- ■ Botón 'Ya verifiqué' entra a la app después de verificar
- ■ Olvidé contraseña envía email de restablecimiento
- ■ Cambiar contraseña desde Perfil funciona
- ■ Logout limpia el backstack y vuelve al login

✔ Permisos y inicio

- ■ Primera vez: PermissionActivity con tarjetas explicativas
- ■ Sin RECORD_AUDIO: no puede continuar
- ■ Con permisos: va a elegir número o a la app

✔ Teclado de marcado

- ■ Se ven los 12 botones (0-9, *, borrar) — CORREGIDO
- ■ Botón Llamar verde siempre visible
- ■ Marcar número y llamar a usuario registrado
- ■ Llamar a número inexistente muestra error

✔ Llamadas VoIP

- ■ Llamada conecta con audio bidireccional en 2 dispositivos
- ■ Ringtone suena en el receptor (pantalla bloqueada y en background)
- ■ Ringtone para al contestar
- ■ Botón Silenciar apaga el micrófono propio
- ■ Botón Altavoz cambia al speaker
- ■ Colgar termina la llamada en ambos dispositivos
- ■ Historial se actualiza al volver a tab Recientes — CORREGIDO
- ■ Rotar pantalla durante llamada NO crashea — CORREGIDO

✔ Config Llamadas

- ■ Botón de ajustes en tab Llamadas abre config — CORREGIDO
- ■ Selector de tono de llamada funciona en Android 13+
- ■ SeekBar ajusta el volumen del ringtone

✔ Mensajes

- ■ Enviar mensaje: aparece ✓ gris (entregado)
- ■ El otro abre el chat: tick se vuelve ✓✓ azul (leído)
- ■ Sin confirmaciones de lectura habilitadas: no aparece tick azul
- ■ Lista de conversaciones refresca al volver a la pantalla — CORREGIDO
- ■ Conversaciones no se duplican — CORREGIDO (clave canónica)

✔ Config Mensajes y Contactos

- ■ Silenciar contacto: no llegan notificaciones de ese contacto
- ■ Programar mensaje: llega a la hora exacta aunque la app esté cerrada

■ Botón Llamar desde Contactos pide permiso si no lo tiene — CORREGIDO
■ Botón Llamar desde Contactos inicia llamada si tiene permiso

✓ Con esta guía v5.0 completa y todos los checks en verde, SimpleCallApp está lista para usar. 21 bugs corregidos, todos los archivos incluidos, google-services.json real integrado.